تیروتیر
آموزشگاه هوش مصنوعی

یاد بگیر • تمرین کن • مهارت پیدا کن

کتاب آموزشی

تیروتیر و پایتون

زبان برنامه‌نویسی تیروتیر + آشنایی با پایتون

برنامه‌نویسی فارسی از صفر



- برنامه‌نویسی منتی و گرافیکی
- آشنایی با پایتون
- ساخت بازی و سرگرمی
- کار با داده‌ها و تحلیل اطلاعات
- پروژه‌های خلاقانه

از صفر تا حرفه‌ای

دکتر داریوش علیپور • دکتر مینوش حیدری

چاپ اول — آبان ۱۴۰۵

منابع و نسخه‌های به‌روز

@AI_and_Philosophy
https://github.com/worker2025/
tirotir.ir

Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)



کتاب آموزشی گام‌به‌گام

تیروتیر

و پایتون

آموزش برنامه‌نویسی از صفر، به زبان فارسی؛ با نکته، یادآوری،
مثال، آزمونک، تمرین خانه و پاسخ‌نامه

دکتر داریوش علیپور · دکتر مینوش حیدری

چاپ -سلام دنیا-

```
print("Hello, world!")
```

بر پایهٔ استاندارد آموزش شغل «برنامه‌نویس تیروتیر» (نسخهٔ ۱ پیشنهادی)
تیروتیر ۲٫۰٫۲ · افزونهٔ VS Code ۲٫۰٫۶ · مخزن رسمی github.com/worker2025/tirotir

شناسنامهٔ کتاب

عنوان کتاب	زبان برنامه‌نویسی تیروتیر + آشنایی با پایتون — برنامه‌نویسی فارسی از صفر
نویسندگان	دکتر داریوش علیپور دکتر مینوش حیدری
سال انتشار	چاپ اول — آبان ۱۴۰۵
نوبت چاپ و ویرایش	اول — آبان ۱۴۰۵
نسخه‌های به‌روز بعدی	کانال تلگرام «فلسفه هوش مصنوعی»: @AI_and_Philosophy گیت‌هاب: https://github.com/worker2025/ وب‌سایت: tirotir.ir

مجوز انتشار

این کتاب با مجوز زیر منتشر شده است:

Creative Commons Attribution–NonCommercial 4.0 International (CC BY-NC 4.0)

بر اساس این مجوز:

- مطالعه، نسخه‌برداری، چاپ، اشتراک‌گذاری و توزیع کتاب آزاد است.
- ترجمه، ویرایش، اقتباس و تولید آثار مشتق از کتاب، با ذکر نام نویسنده و صرفاً برای اهداف غیرتجاری، مجاز است.
- استفادهٔ تجاری، از جمله فروش کتاب (چاپی یا الکترونیکی) یا فروش آثار مشتق، بدون اجازهٔ کتبی نویسنده مجاز نیست.
- در تمامی نسخه‌های منتشرشده، نام نویسنده و لینک نسخهٔ به‌روزرسانی باید حفظ شود:
  tirotir.ir · https://github.com/worker2025/ · @AI_and_Philosophy

متن کامل مجوز:
https://creativecommons.org/licenses/by-nc/4.0

همهٔ مثال‌های این کتاب با **تیروتیر ۲٫۰٫۲** اجرا و خروجی‌شان از اجرای واقعی برداشته شده است. مرجع اصلی زبان: github.com/worker2025/tirotir.

پیش‌گفتار و راهنمای استفاده

زبان، ابزار اندیشیدن است و برنامه‌نویسی، زبانی برای گفتگو با ماشین. آموزش برنامه‌نویسی برای فارسی‌زبانان تاکنون با مانعی دوگانه همراه بوده است: یادگیرنده ناگزیر است هم‌زمان با مفاهیم بنیادین الگوریتم و منطق، با واژگان و پیام‌های خطای بیگانه نیز دست‌وپنجه نرم کند. تیروتیر با دستورها و پیام‌های خطای فارسی کوشیده است این مانع را بردارد و توجه یادگیرنده را به خودِ تفکر الگوریتمی معطوف کند.

کتاب حاضر بر پایهٔ «استاندارد آموزش شغل برنامه‌نویس تیروتیر» (نسخهٔ پیشنهادی) و در چهارده فصل تدوین شده است؛ از نخستین برنامه و مفهوم متغیر آغاز می‌کند و با شرط، حلقه، تابع، ساختارهای داده، گرافیک، صدا، رابط گرافیکی و ساخت برنامهٔ اجرایی ادامه می‌یابد. هر فصل شامل نکته، یادآوری، مثال اجراشده، آزمونک و تمرین خانه است و پاسخ‌ها در انتهای کتاب آمده‌اند. همهٔ مثال‌ها با **تیروتیر ۲٫۰٫۲** اجرا و خروجی‌ها از اجرای واقعی برداشته شده‌اند. در پایان مباحث مهم، بخش «**مطالعهٔ آزاد: پایتون**» همان مفهوم را در یکی از پرکاربردترین زبان‌های جهان عرضه می‌کند تا مسیر یادگیری پس از تیروتیر هموار باشد.

نویسندگان بر این باورند که برنامه‌نویسی با اجرا کردن و خطا دیدن آموخته می‌شود، نه با خواندنِ صرف؛ از این رو از خواننده می‌خواهند مثال‌ها را خود تایپ و اجرا کند. بی‌تردید این اثر از کاستی خالی نیست. پیشنهادها و نقدهای خوانندگان و مربیان گرامی را، از راه‌های ارتباطی ذکرشده در شناسنامهٔ کتاب، با سپاس می‌پذیریم و در ویرایش‌های بعدی به‌کار می‌بندیم.

دکتر داریوش علیپور · دکتر مینوش حیدری — آبان ۱۴۰۵

راهنمای استفاده از کتاب

هر فصل از چه ساخته شده؟

نکته
ترفندها و نکته‌های مفید.

یادآوری
چیزی که نباید فراموش کنید.

توجه
خطرها، محدودیت‌ها و اشتباه‌های رایج.

فعالیت بدون رایانه
بازی و نقش‌آفرینی با کارت و کاغذ.

مطالعهٔ آزاد: پایتون
همان ایده در پایتون.

مثال‌های گیتهاب
فایل‌های آمادهٔ مخزن رسمی + QR.

هر فصل با «در پایان این فصل می‌توانید» شروع می‌شود و با یک **آزمونک** پنج‌پرسشی و چهار **تمرین خانه** تمام می‌شود. پاسخ همهٔ آن‌ها، همراه با کدِ اجراشده و خروجی، در **پاسخ‌نامهٔ انتهای کتاب** است.

چطور بخوانیم؟

- کد را فقط نخوانید؛ آن را **خودتان تایپ کنید** و اجرا کنید. خطا بخشی طبیعی از یادگیری است.
- پیش از دیدن خروجی، آن را **حدس بزنید**.
- فعالیت‌های «بدون رایانه» را با دوستان انجام دهید؛ ایده‌ها اول روی کاغذ روشن می‌شوند.

- هر جلسه ۹۰ تا ۱۲۰ دقیقه کار کنید و دست‌کم یک استراحت ۱۰ دقیقه‌ای داشته باشید. پس از هر ۲۰ تا ۳۰ دقیقه، چشمتان را از نمایشگر بردارید.
- فصل‌های ۱ و ۲ نخست‌اند؛ فصل‌های ۳ تا ۸ هستهٔ اصلی‌اند؛ فصل‌های ۹ تا ۱۱ (گرافیک، صدا، پنجره) به هر ترتیبی قابل‌خواندن‌اند؛ فصل ۱۲ (خطا) را در طول دوره تمرین کنید؛ فصل ۱۳ پس از چند برنامهٔ کامل و فصل ۱۴ آخر می‌آید.

نسخه‌ها را یکی نگیرید

این کتاب با **تیروتیر ۲٫۰٫۲** و **افزونهٔ VS Code نسخهٔ ۲٫۰٫۶** نوشته شده است (نسخهٔ افزونه جدا شماره‌گذاری می‌شود). اگر `tirotir version` شما نسخهٔ ۱٫۱٫۰ نشان داد، یک نصب قدیمی دارید؛ فصل ۱ را ببینید.

یک قاعدهٔ طلایی

در نسخهٔ ۲، متن ثابت (`-متن-`) را فقط اول یک عبارت بنویسید، و بعد از خط‌تیرهٔ آغازین بلافاصله حرف بیاید (نه فاصله). جزئیات در فصل ۳.

فهرست مطالب

پیش‌گفتار و راهنمای استفاده ۳

بخش اول: شایستگی‌های دوره

فصل ۱ آشنایی با برنامه‌نویسی و محیط تیروتیر ۶

فصل ۲ کار با VS Code، افزونهٔ تیروتیر و اسنیپت‌ها ۱۱

فصل ۳ متغیر، ورودی و خروجی ۱۴

فصل ۴ محاسبات و عبارت‌های ریاضی ۲۰

فصل ۵ شرط و تصمیم‌گیری ۲۴

فصل ۶ حلقه و تکرار ۳۰

فصل ۷ تابع و استفادهٔ مجدد از کد ۳۷

فصل ۸ فهرست و فرهنگ ۴۳

فصل ۹ گرافیک و نقاشی با برنامه ۵۰

فصل ۱۰ تولید صدا و موسیقی ساده ۵۶

فصل ۱۱ ساخت برنامهٔ دارای پنجره (رابط گرافیکی) ۶۱

فصل ۱۲ بررسی برنامه و رفع خطا ۶۷

فصل ۱۳ ساخت برنامهٔ اجرایی با Tirotir Builder ۷۴

فصل ۱۴ پروژهٔ چندفایلی، قابلیت‌ها و ارائهٔ پروژهٔ پایانی ۷۸

پیوست‌ها

پیوست آ دستورهای خط فرمان تیروتیر ۸۵

پیوست ب برگهٔ مقایسه: تیروتیر و پایتون ۸۶

پیوست پ واژه‌نامه ۸۷

پیوست ت محدودیت‌های نسخهٔ ۲٫۰٫۲ در یک نگاه ۸۸

پیوست ث منابع و پیوندهای مهم ۸۹

پاسخ‌نامه

پاسخ آزمونک‌ها و تمرین‌های خانهٔ همهٔ فصل‌ها ۹۰

فصل ۱

آشنایی با برنامه‌نویسی و محیط تیروتیر

در پایان این فصل می‌توانید:

- توضیح دهید برنامه، الگوریتم و دستور چیست
- تیروتیر ۲ را نصب و با `tirotir version` بررسی کنید
- نخستین برنامه (سلام دنیا) را بنویسید، ذخیره و اجرا کنید
- تفاوت «اجرا» و «بررسی» برنامه را بگویید
- با محیط تعاملی `tirotir repl` کار کنید

۱-۱ برنامه چیست؟

رایانه خودش نمی‌داند چه کار کند؛ باید به او بگوییم. فهرستی از دستورهای مرتب که به رایانه می‌گوییم «**برنامه**» نام دارد. مثل دستور پخت یک غذا یا مسیر رفتن از خانه تا مدرسه: اول این کار، بعد آن کار. روشِ گام‌به‌گام برای حل یک مسئله را «**الگوریتم**» می‌گویند و هر گامِ آن، یک «**دستور**» است.

فعالیت بدون رایانه

ربات و مسیر. یک نفر نقش «ربات» را دارد و چشم‌هایش را می‌بندد. دوستش فقط با دستورهای «یک قدم جلو»، «بچرخ به راست» و «بایست» او را از میز تا در کلاس راهنمایی می‌کند. اگر ربات به صندلی خورد، یعنی برنامه خطا داشت! خطا را پیدا کنید و دستورها را درست کنید. نقش‌ها را عوض کنید.

۱-۲ تیروتیر چیست؟

تیروتیر یک زبان برنامه‌نویسی فارسی است. یعنی دستورها، نام‌ها و حتی پیام‌های خطا به فارسی‌اند و لازم نیست پیش از شروع، انگلیسی بلد باشید. تیروتیر یک «**مفسر**» دارد: برنامهٔ کوچکی که فایل شما را خط‌به‌خط می‌خواند و انجام می‌دهد. فایل‌های تیروتیر با پسوند `.t` ذخیره می‌شوند؛ مثلاً `hello.t`.

نکته

نسخهٔ این کتاب تیروتیر **۲٫۰٫۲** (نسخهٔ پایدار فعلی) و افزونهٔ VS Code نسخهٔ **۲٫۰٫۶** است. این دو عدد یکی نیستند: نسخهٔ زبان و نسخهٔ افزونه جدا شماره‌گذاری می‌شوند.

۱-۳ اجزای رایانه‌ای که به آن‌ها نیاز داریم

- **صفحه‌کلید:** برای تایپ برنامه. باید بتوانید فارسی تایپ کنید.
- **موس:** برای کلیک و انتخاب.
- **نمایشگر:** برنامه و نتیجهٔ آن را می‌بینیم.
- **فایل و پوشه:** هر برنامه یک فایل است؛ فایل‌های مرتبط را در یک پوشه نگه می‌داریم.

ایمنی و سلامت

صاف بنشینید، فاصلهٔ چشم تا نمایشگر را حدود یک وجب و نیم نگه دارید و پس از هر ۲۰ تا ۳۰ دقیقه کار، کمی استراحت کنید. به کابل‌ها و پریزهای برق بدون اجازهٔ مربی دست نزنید و پس از کار، رایانه را خاموش کنید.

۱-۴ نصب تیروتیر

تیروتیر ۲ یک بستهٔ پایتون است و به **Python ۳٫۱۱ یا بالاتر** نیاز دارد. در یک ترمینال (PowerShell یا Command Prompt در ویندوز) این دستور را بزنید:

خط فرمان

```
python -m pip install tirotir==2.0.2
```

بستهٔ آفلاین (`wheel`) هم در صفحهٔ «Releases» مخزن رسمی هست. پس از نصب، بررسی کنید:

خط فرمان؛ خط دوم پاسخ رایانه است

```
tirotir version
Tirotir 2.0.2
```

توجه

فایل `Tirotir-1.1.0-Setup.exe` نصب‌کنندهٔ **تاریخی نسخهٔ ۱** است و نسخهٔ ۲ را نصب نمی‌کند. اگر خروجی `tirotir version` شما `1.1.0` بود، یک نصب قدیمی دارید؛ روش بالا را دوباره انجام دهید.

۱-۵ نخستین برنامه

یک فایل به نام `hello.t` بسازید و این یک خط را در آن بنویسید. متن در تیروتیر بین دو **خط‌تیره** می‌آید: `-متن-` (نه گیومه).

hello_world.t — examples2/basics/hello_world.t

```
# نخستین برنامهٔ تیروتیر
چاپ -سلام دنیا-
```

خروجی

```
سلام دنیا
```

برای اجرا، در ترمینال و در همان پوشه این دستور را بزنید:

خط فرمان

```
tirotir run hello.t
```

سه برنامهٔ بعدی همین مسیر را دارند. چند خط «چاپ» پشت سر هم، خط‌به‌خط اجرا می‌شود:

print_messages.t — examples2/basics/print_messages.t

```
چاپ -به تیروتیر خوش آمدید-
چاپ -این زبان برای یادگیری و ساخت برنامه است-
چاپ -هر فایل تیروتیر با پسوند t ذخیره می‌شود-
```

خروجی

```
به تیروتیر خوش آمدید
این زبان برای یادگیری و ساخت برنامه است
هر فایل تیروتیر با پسوند t ذخیره می‌شود
```

یادآوری

هر دستور در یک خط جدا نوشته می‌شود. ترتیب مهم است: رایانه خطوط را از بالا به پایین اجرا می‌کند.

۱-۶ اجرا یا بررسی؟

دستور `tirotir run` برنامه را **اجرا** می‌کند. دستور `tirotir check` فقط **بررسی** می‌کند که برنامه از نظر نوشتن درست است یا نه، بدون این‌که آن را اجرا کند:

hello.t

```
چاپ -سلام دنیا-
```

نتیجهٔ بررسی

```
برنامه صحیح است.
```

نکته

«برنامه صحیح است» یعنی نوشتنِ برنامه درست است؛ **نه این‌که حتماً همه‌چیز موقع اجرا هم درست پیش می‌رود**. در فصل ۱۲ می‌بینید که بعضی خطاها فقط موقع اجرا پیدا می‌شوند.

۱-۷ محیط تعاملی (REPL)

اگر فقط `tirotir repl` را بزنید، رایانه منتظر می‌ماند و هر دستوری که تایپ کنید بلافاصله اجرا می‌کند؛ برای آزمایش‌های کوچک عالی است. برای خروج، کلمهٔ **خروج** را بنویسید.

خط فرمان

```
tirotir repl
```

گفتگو با محیط تعاملی

```
REPL تیروتیر ۲٫۰ — برای خروج «خروج» بنویسید.
تیروتیر> چاپ ۱ + ۱
2
تیروتیر> خروج
```

مثال‌های آمادهٔ این فصل در گیتهاب

پوشهٔ **examples2/basics** را در مخزن رسمی ببینید و فایل‌ها را بخوانید و اجرا کنید:

```
basics/hello_world.t
basics/print_messages.t
https://github.com/worker2025/tirotir/tree/main/examples2/basics
```

مطالعهٔ آزاد: پایتون

پایتون یکی از معروف‌ترین زبان‌های برنامه‌نویسی دنیاست. (تیروتیر هم یک بستهٔ پایتون است و مفسرش با پایتون نوشته شده.) در پایتون دستور چاپ `print` است و متن بین گیومه می‌آید. این برنامه را در فایل `hello.py` بنویسید و با `python hello.py` اجرا کنید:

```
print("Hello, world!")
print("سلام دنیا")
```

خروجی:

```
Hello, world!
سلام دنیا
```

دو تفاوت: در پایتون دستورها انگلیسی‌اند و متن بین گیومه است. ولی ایدهٔ «هر دستور در یک خط و اجرا از بالا به پایین» دقیقاً همان است.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. به فهرست مرتب دستورهایی که رایانه باید انجام دهد چه می‌گویند؟

الف) برنامه ب) نمایشگر ج) پوشه د) موس

۲. پسوند فایل‌های تیروتیر چیست؟

الف) .py ب) .t ج) .exe د) .doc

۳. کدام دستور برنامه را بدون اجرا فقط بررسی می‌کند؟

الف) tirotir run hello.t ب) tirotir version ج) tirotir check hello.t د) tirotir repl

۴. متن در تیروتیر بین چه علامتی نوشته می‌شود؟

الف) گیومه " " ب) پرانتز ( ) ج) خط‌تیره - - د) کروشه [ ]

۵. برای خروج از `tirotir repl` چه بنویسیم؟

الف) بیرون ب) خروج ج) پایان د) ببند

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. آسان برنامه‌ای بنویسید که سه خط دربارهٔ خودتان (نام، شهر و یک علاقه‌مندی) چاپ کند.
2. آسان برنامه‌ای بنویسید که کلمهٔ «تیروتیر» را چهار بار، هر بار در یک خط، چاپ کند (با چهار دستور چاپ).
3. متوسط بالای برنامهٔ تمرین ۱ یک خط توضیح (کامنت) بنویسید که با علامت # شروع شود. بعد برنامه را اجرا کنید. آیا خط توضیح در خروجی آمد؟
4. چالشی برنامه‌ای بنویسید که یک «آگهی» سه‌خطی چاپ کند: خط اول عنوان، خط دوم توضیح، خط سوم شمارهٔ تماس (فرضی). بعد آن را هم اجرا کنید و هم با `tirotir check` بررسی کنید.

فصل ۲

کار با VS Code، افزونهٔ تیروتیر و اسنیپت‌ها

در پایان این فصل می‌توانید:

- افزونهٔ TirotirLang را در VS Code نصب کنید
- فایل جدید `.t` بسازید و رنگ‌بندی کد را ببینید
- با اسنیپت‌ها کد آماده درج کنید
- برنامه را با منوی راست‌کلیک اجرا و بررسی کنید
- خروجی را در پنل فارسی راست‌به‌چپ بخوانید

۲-۱ چرا ویرایشگر کد؟

می‌شود برنامه را در Notepad هم نوشت، ولی یک **ویرایشگر کد** مثل **VS Code** کار را خیلی راحت‌تر می‌کند: کلمه‌های مهم را رنگی نشان می‌دهد، تورفتگی را خودکار درست می‌کند و با یک کلیک برنامه را اجرا می‌کند.

۲-۲ نصب افزونهٔ TirotirLang

افزونهٔ رسمی تیروتیر با نام `tirotirLang` و فایل `tirotirlang-2.0.6.vsix` عرضه می‌شود. دو راه نصب:

1. در VS Code کلیدهای `Ctrl+Shift+P` را بزنید (پالت فرمان) و گزینهٔ نصب از فایل VSIX را انتخاب کنید و فایل را نشان دهید.
2. یا در خط فرمان:

```
خط فرمان
code --install-extension tirotirlang-2.0.6.vsix
```

حالا هر فایل `.t` را که باز کنید، رنگ‌بندی خودکار فعال می‌شود: کلیدواژه‌ها، عددها، متن‌ها و توضیح‌ها هر کدام رنگ خودشان را دارند.

نکته

اگر VS Code فرمان `tirotir` را پیدا نکرد، در بخش Settings مقدار `tirotirLang.command` را برابر مسیر کامل فرمان tirotir بگذارید (پیش‌فرض آن همان `tirotir` است).

۲-۳ ساخت فایل و اسنیپت‌ها

با دکمهٔ تیروتیر در نوار وضعیت پایین VS Code یک فایل جدید `.t` می‌سازید؛ نام آن خودکار انتخاب می‌شود (مثل `program-001.t`). «اسنیپت» یعنی تکه‌کدِ آماده. روش کار: **پیشوند را بنویسید، `Ctrl+Space` بزنید، بعد `Tab`**.

پیشوند	کاری که می‌کند	پیشوند	کاری که می‌کند
سلام	چاپ پیام ساده (سلام دنیا)	تابع	ساخت تابع و بازگرداندن مقدار
متغیر	ساخت متغیر	فهرست	ساخت فهرست نمونه
چاپ	چاپ متن	فرهنگ	ساخت فرهنگ نمونه
اگر	شرط دوشاخه (اگر/وگرنه)	ورودی	گرفتن و چاپ ورودی
تکرار	تکرار به تعداد مشخص	پنجره	ساخت پنجرهٔ GUI
تاوقتی	حلقهٔ شرطی	نت	دستور نت صوتی
برای	پیمایش فهرست	مربع	رسم مربع

۲-۴ اجرا، بررسی و AST

روی هر فایل `.t` راست‌کلیک کنید. این فرمان‌ها را می‌بینید:

- **TirotirLang: اجرای فایل** — برنامه را اجرا می‌کند.
- **TirotirLang: بررسی فایل** — برنامه را فقط بررسی می‌کند.
- **TirotirLang: نمایش AST** — ساختار برنامه را به‌صورت «درخت» نشان می‌دهد (فقط برای آشنایی).
- **اجرای زنده در پنل فارسی راست‌به‌چپ** — خروجی فارسی را درست و راست‌به‌چپ نشان می‌دهد؛ اگر برنامه «بخوان» داشته باشد، یک کادر ورودی می‌دهد.
- **نمایش Graphics زنده**، **پخش Audio** و **اجرای GUI** — برای فصل‌های ۹، ۱۰ و ۱۱.

همین کارها از خط فرمان هم شدنی است. دستور `tirotir ast` درخت برنامه را به شکل JSON نشان می‌دهد؛ این نمونه برای برنامه‌ای با دو خط است:

a.t

```
بگذار الف برابر ۳
چاپ الف
```

خط فرمان

```
tirotir ast a.t
```

توجه

در نسخهٔ ۲ هنوز **LSP کامل**، تکمیل خودکار پیشرفته، راهنمای شناور و **اشکال‌زدای حرفه‌ای** نیست. اگر انتظار این‌ها را در VS Code داشتید، نگران نباشید: خطای شما نیست.

ایمنی و سلامت

اندازهٔ فونت و روشنایی ویرایشگر را طوری تنظیم کنید که چشم‌تان اذیت نشود. پس از پایان کار ویرایشگر و ترمینال را ببندید.

مطالعهٔ آزاد: پایتون

پایتون هم ویرایشگر مخصوص دارد، ولی VS Code با افزونهٔ **Python** برای آن هم عالی است. فایل `.py` را باز کنید و دکمهٔ «Run» بالای پنجره را بزنید. اسنیپت و رنگ‌بندی همین ایده را در پایتون هم دارد. ببینید همان «سلام دنیا» چطور رنگی دیده می‌شود:

```
# این یک توضیح است
name = "Sara"
print("Hello", name)
```

خروجی:

```
Hello Sara
```

نکتهٔ مشترک: خط‌هایی که با `#` شروع شوند در هر دو زبان توضیح‌اند.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. افزونهٔ رسمی تیروتیر در VS Code چه نام دارد؟

الف) tirotirLang ب) PyCode ج) FarsiPad د) TirotirAI

۲. برای درج اسنیپت، پس از نوشتن پیشوند چه کلیدهایی می‌زنیم؟

الف) Ctrl+Space سپس Tab ب) Alt+F4 ج) Enter دوبار د) Shift+Delete

۳. پیشوند اسنیپتی که یک شرط «اگر / وگرنه» می‌سازد چیست؟

الف) تکرار ب) اگر ج) برای د) نت

۴. فرمان «نمایش AST» چه چیزی نشان می‌دهد؟

الف) صدای برنامه ب) ساختار درختی برنامه ج) تصویر SVG د) پنجرهٔ GUI

۵. کدام مورد هنوز در نسخهٔ ۲ افزونه کامل نیست؟

الف) رنگ‌بندی ب) اجرای فایل ج) اشکال‌زدای حرفه‌ای د) اسنیپت‌ها

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. **آسان** در VS Code افزونه را نصب کنید و فایلی بسازید. با اسنیپت «سلام» برنامهٔ سلام دنیا را درج کنید و با راست‌کلیک اجرا کنید. (تمرین عملی روی رایانه است؛ پاسخ، همان برنامهٔ زیر است.)
2. **متوسط** با اسنیپت‌های «متغیر» و «چاپ» برنامه‌ای بسازید که نام شما را در یک متغیر نگه دارد و چاپ کند.
3. **متوسط** برنامهٔ تمرین ۲ را با «بررسی فایل» (راست‌کلیک) بررسی کنید. نتیجه چه شد؟
4. **چالشی** با دست‌کم پنج اسنیپت مختلف (مثلاً سلام، متغیر، چاپ، اگر، تکرار) یک برنامهٔ چندخطی بسازید و آن را اجرا و بررسی کنید.

فصل ۳

متغیر، ورودی و خروجی

در پایان این فصل می‌توانید:

- متغیر را به‌عنوان «جعبهٔ نام‌دار» توضیح دهید
- متغیر عدد و متن بسازید و چاپ کنید
- از کاربر ورودی بگیرید (با «بخوان»)
- مقدار متغیر را تغییر دهید و اثرش را ببینید
- برنامهٔ «معرفی خودم» بنویسید

۳-۱ متغیر؛ جعبهٔ نام‌دار

گاهی باید چیزی را به خاطر بسپاریم؛ مثلاً سنِ کسی یا نمرهٔ یک درس. «**متغیر**» جعبه‌ای است که روی آن اسمی نوشته‌ایم و داخلش یک مقدار می‌گذاریم. هر وقت اسم را بگوییم، مقدارِ داخل جعبه را می‌گیریم. در تیروتیر این‌طور می‌سازیم:

variables.t

```
بگذار سن برابر ۲۰
بگذار نام برابر -سارا-
چاپ نام
چاپ سن
```

خروجی

```
سارا
20
```

خواندنش آسان است: «بگذار **سن** برابر ۲۰». نام‌ها می‌توانند فارسی باشند.

فعالیت بدون رایانه

جعبه‌های نام‌دار. چند جعبهٔ کوچک (یا لیوان کاغذی) بردارید و روی هر کدام برچسب بچسبانید: «سن»، «نام»، «شهر». روی تکه‌کاغذهای کوچک مقدارها را بنویسید و در جعبه‌ها بگذارید. حالا از یک نفر بخواهید: «محتوای جعبهٔ شهر را بگو». بعد مقدار داخل جعبهٔ «سن» را عوض کنید. متغیر هم دقیقاً همین‌طور است.

۳-۲ نوع داده: عدد و متن

در جعبه‌ها دو نوع چیز می‌گذاریم: **عدد** (مثل ۲۰ یا ۱۲٫۵) و **متن** (مثل `-سارا-`). تفاوتشان مهم است: با عدد می‌شود حساب کرد، با متن نه. کدی که سه نوع مقدار را نگه می‌دارد، این است:

multiple_values.t examples2/variables/multiple_values.t

```
بگذار عنوان برابر -کتاب تیروتیر-
بگذار صفحه‌ها برابر ۱۲۸
بگذار آماده برابر درست
چاپ عنوان
چاپ صفحه‌ها
چاپ آماده
```

خروجی

```
کتاب تیروتیر
128
True
```

نکته

«درست» و «نادرست» هم دو مقدار ویژه‌اند. توجه کنید خروجی به‌شکل `True` (درست) و `False` (نادرست) چاپ می‌شود؛ مفسر تیروتیر روی پایتون ساخته شده و همین رسم را دارد.

۳-۳ انتخاب نام مناسب

- نام باید معنی داشته باشد: `سن` بهتر از `الف`.
- نام می‌تواند فارسی یا انگلیسی باشد؛ فاصله ندارد. اگر چند کلمه‌ای بود، با خطزیر یا نیم‌فاصله بنویسید: `نمره_درس`.
- دقت کنید املای کلیدواژه‌ها را درست بنویسید: «بگذار»، «برابر»، «چاپ».

۳-۴ تغییر مقدار

مقدار داخل جعبه را می‌شود عوض کرد. دوباره «بگذار» می‌نویسیم و در آن از مقدار قبلی هم می‌توانیم استفاده کنیم:

change_value.t examples2/variables/change_value.t

```
بگذار نام برابر -مینا-
بگذار امتیاز برابر ۱۰
چاپ نام
چاپ امتیاز
بگذار امتیاز برابر امتیاز + ۵
چاپ امتیاز
```

خروجی

```
مینا
10
15
```

یادآوری

رایانه خط‌ها را به‌ترتیب اجرا می‌کند. وقتی به `بگذار امتیاز برابر امتیاز + ۵` می‌رسد، اول سمت راست را حساب می‌کند (۱۰ + ۵) و بعد نتیجه را در جعبهٔ «امتیاز» می‌گذارد.

۳-۵ ورودی و خروجی

خروجی یعنی نمایش چیزی به کاربر (با «چاپ»). **ورودی** یعنی گرفتن چیزی از کاربر؛ این کار را «**بخوان**» می‌کند:

ask_name.t — examples2/input/ask_name.t

```
چاپ -نامت چیست؟-
بخوان
چاپ -از آشنایی با تو خوشحالم.-
```

```
خروجی
ورودی تایپ‌شده: سارا
نامت چیست؟
از آشنایی با تو خوشحالم.
```

در این برنامه «بخوان» منتظر می‌ماند تا کاربر چیزی تایپ کند و Enter بزند؛ ولی مقدار تایپ‌شده را جایی نگه نمی‌داریم. برای استفاده از آن، در متغیر بریزید:

summary.t — examples2/output/summary.t

```
بگذار نام برابر بخوان
بگذار امتیاز برابر ۱۹
چاپ -نام: - + نام
چاپ -امتیاز: - + به_متن با امتیاز
```

```
خروجی
ورودی تایپ‌شده: علی
نام: علی
امتیاز: 19
```

نکته

برای چسباندن دو متن به هم از `+` استفاده می‌کنیم. اما عدد را نمی‌شود مستقیم به متن چسباند؛ ابتدا باید با `... به_متن با` آن را به متن تبدیل کرد (مثل «امتیاز» در برنامهٔ بالا).

توجه

قاعدهٔ مهم دربارهٔ متن: در نسخهٔ ۲، متن ثابتِ بین خطتیره را فقط می‌توان **اول** یک عبارت نوشت (مثلاً `چاپ -سلام - + نام`)؛ بعد از علامت `+` نمی‌شود متن ثابت گذاشت. راه‌حل ساده: متن‌های ثابت بعدی را اول در یک متغیر بریزید و بعد آن متغیر را بچسبانید (مانند برنامهٔ «معرفی خودم» در پایین). همچنین بعد از خطتیرهٔ آغازین بلافاصله حرف بیاید، نه فاصله: `- سلام` درست است ولی `-سلام -` نه. این یک «محدودیت شناخته‌شده» (KI-1) است و در نسخه‌های بعدی بهتر می‌شود.

توجه

«بخوان» همیشه یک **متن** برمی‌گرداند؛ حتی اگر کاربر عدد تایپ کند. برای حساب‌کردن با آن، باید `... به_عدد با` را به کار ببرید (فصل ۴).

۳-۶ برنامهٔ «معرفی خودم»

intro.t

```
چاپ -نام شما چیست؟-
بگذار نام برابر بخوان
چاپ -اهل کدام شهر هستید؟-
بگذار شهر برابر بخوان
بگذار الف برابر -سلام -
بگذار ب برابر -، به شهر -
بگذار پ برابر -. خوش آمدی-
چاپ الف + نام + ب + شهر + پ
```

خروجی

ورودی تایپ‌شده: **نرگس ، شیراز**

```
نام شما چیست؟
اهل کدام شهر هستید؟
سلام نرگس، به شهر شیراز. خوش آمدی
```

مثال‌های آمادهٔ این فصل در گیتهاب

این فایل‌ها در پوشهٔ **examples2** مخزن رسمی هستند؛ بخوانید و اجرا کنید:

```
variables/multiple_values.t
variables/change_value.t
input/ask_name.t
output/summary.t
```

https://github.com/worker2025/tirotir/tree/main/examples2

مطالعهٔ آزاد: پایتون

در پایتون هم متغیر جعبهٔ نام‌دار است؛ فقط نیازی به «بگذار» نیست و از `=` استفاده می‌شود. «درست/نادرست» در پایتون `True` و `False` است. ورودی را `input` می‌گیرد، و آن هم همیشه متن برمی‌گرداند:

```
age = 20
name = "Sara"
ready = True
print(name)
print(age)
print(ready)
age = age + 5
print(age)
city = input()
print("Hello " + name + " from " + city)
```

خروجی:

```
Sara
20
True
25
Hello Sara from Shiraz
```

در پایتون برای چسباندن متن و عدد باید عدد را با `str(age)` به متن تبدیل کرد؛ درست مثل `به_متن` در تیروتیر.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. «بگذار سن برابر ۲۰» چه کاری می‌کند؟

الف) سن را چاپ می‌کند ب) مقدار ۲۰ را در متغیری به نام سن می‌گذارد ج) سن را از کاربر می‌پرسد د) برنامه را تمام می‌کند

۲. کدام یک «متن» است؟

الف) ۲۰ ب) -سارا- ج) درست د) ۳٫۵

۳. دستور «بخوان» چه چیزی را برمی‌گرداند؟

الف) همیشه عدد ب) همیشه متن ج) درست یا نادرست د) هیچ چیز

۴. پس از اجرای این دو خط چه چاپ می‌شود؟

`بگذار الف برابر ۴`

`بگذار الف برابر الف + ۱` و بعد `چاپ الف`

الف) ۴ ب) ۱ ج) ۵ د) ۴۱

۵. برای چسباندن عدد به متن در چاپ از کدام دستور کمک می‌گیریم؟

الف) به‌عدد ب) به‌متن ج) طول د) افزودن

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. آسان — سه متغیر با سه نوع مختلف (متن، عدد، درست/نادرست) بسازید و هر سه را چاپ کنید.
2. آسان — برنامه‌ای بنویسید که نام را از کاربر بگیرد و بنویسد «سلام ...، خوش آمدی».
3. متوسط — برنامهٔ «معرفی خودم» را بنویسید که نام، سن (به‌عنوان متن) و شهر را بپرسد و هر کدام را در یک خط چاپ کند.
4. چالشی — متغیر «امتیاز» را برابر ۱۰ بگذارید، دو بار ۵ به آن اضافه کنید و پس از هر بار مقدارش را چاپ کنید.

فصل ۴

محاسبات و عبارت‌های ریاضی

در پایان این فصل می‌توانید:

- چهار عمل اصلی را در برنامه به‌کار ببرید
- نقش پرانتز و اولویت عملگرها را توضیح دهید
- مساحت، محیط و میانگین را محاسبه کنید
- ورودی را به عدد تبدیل کنید و ماشین‌حساب ساده بسازید

۴-۱ چهار عمل اصلی

رایانه پیش از هر چیز یک ماشین‌حساب است. علامت‌ها: جمع `+`، تفریق `-`، ضرب `*`، تقسیم `/` و باقی‌ماندهٔ تقسیم `%`.

four_operations.t — examples2/math/four_operations.t

```
بگذار الف برابر ۲۴
بگذار ب برابر ۶
چاپ الف + ب
چاپ الف - ب
چاپ الف * ب
چاپ الف / ب
```

خروجی

```
30
18
144
4.0
```

نکته

تقسیم همیشه جواب اعشاری می‌دهد: `۲۴ / ۶` می‌شود `4.0` (نه ۴). اعشار در برنامه با «٫» یا نقطه نوشته می‌شود: `۳٫۱۴` .

۴-۲ فاصله دور عملگرها و اولویت

مثل ریاضی مدرسه، **ضرب و تقسیم پیش از جمع و تفریق** انجام می‌شود، و هر چه داخل پرانتز باشد اول حساب می‌شود:

calc.t

```
چاپ ۲ + ۳ * ۴
چاپ (۲ + ۳) * ۴
چاپ ۵ - ۳ + ۲ - ۱
چاپ ۱۰ % ۴
```

خروجی

```
14
20
3
2
```

یادآوری

دور عملگرها فاصله بگذارید: `۱ - ۲ + ۳ - ۵` . این کار خواندن را آسان می‌کند، و چون خط‌تیره هم برای «متن» به کار می‌رود و هم برای «منها»، رعایت فاصله جلوی ابهام را می‌گیرد. در نسخهٔ ۲ عبارت فشردهٔ `۱-۲+۳-۵` هم برابر ۳ است (در نسخهٔ ۱٫۱۰ برابر ۵ بود).

۴-۳ ترکیب متن و عدد

examples2/math/rectangle_area.t — rectangle_area.t

```
بگذار طول برابر ۸
بگذار عرض برابر ۵
بگذار مساحت برابر طول * عرض
چاپ "مساحت مستطیل: " + به_متن با مساحت
```

خروجی

```
مساحت مستطیل: 40
```

محیط همان مستطیل `(طول + عرض) * ۲` است. و میانگین چند عدد، جمع آن‌ها تقسیم بر تعدادشان:

average.t

```
بگذار الف برابر ۱۸
بگذار ب برابر ۱۶
بگذار پ برابر ۲۰
بگذار میانگین برابر (الف + ب + پ) / ۳
چاپ میانگین
```

خروجی

```
18.0
```

۴-۴ ماشین‌حساب ساده

چون «بخوان» متن می‌دهد، ورودی را با `به_عدد با ...` به عدد تبدیل می‌کنیم (نتیجه یک عدد اعشاری است):

toNumber.t

بگذار متن۱ **برابر** -۱۲-

بگذار الف **برابر به_عدد با** متن۱

چاپ الف + ۱

خروجی

```
13.0
```

توجه

`به_عدد` نتیجه را **اعشاری** می‌کند: ۱۲ می‌شود `12.0` و ۱۲ + ۱ می‌شود `13.0` . این طبیعی است.

ایمنی و سلامت

هنگام تایپ مچ دست را صاف نگه دارید و ارتفاع صندلی را تنظیم کنید. هر چند دقیقه چشمتان را از نمایشگر بردارید.

مثال‌های آمادهٔ این فصل در گیتهاب

پوشهٔ **examples2/math** را در مخزن رسمی ببینید و فایل‌ها را بخوانید و اجرا کنید:

```
math/four_operations.t
math/rectangle_area.t
https://github.com/worker2025/tirotir/tree/main/examples2/math
```

مطالعهٔ آزاد: پایتون

در پایتون هم همین عمل‌ها هست و نتیجه‌ها یکسان است. پایتون علاوه بر آن `//` (تقسیم صحیح) و `**` (توان) دارد:

```
print(24 / 6)
print(2 + 3 * 4)
print((2 + 3) * 4)
print(10 % 4)
print(7 // 2)
print(2 ** 3)
text = "12"
number = float(text)
print(number + 1)
```

خروجی:

```
4.0
14
20
2
3
8
13.0
```

`float(...)` در پایتون کار «به_عدد» را می‌کند. دقت کنید در پایتون هم `۲۴ / ۶` می‌شود `4.0` .

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. حاصل `چاپ ۲ + ۳ * ۴` چیست؟

الف) ۲۰ ب) ۱۴ ج) ۹ د) ۲۴

۲. حاصل `چاپ (۲ + ۳) * ۴` چیست؟

الف) ۲۰ ب) ۱۴ ج) ۹ د) ۱۲

۳. `چاپ ۲۴ / ۶` چه چاپ می‌کند؟

الف) ۴ ب) 4.0 ج) ۴٫۵ د) خطا

۴. علامت «باقی‌ماندهٔ تقسیم» کدام است؟

الف) / ب) * ج) % د) +

۵. برای تبدیل متنِ ورودی به عدد از چه دستوری استفاده می‌کنیم؟

الف) به_متن ب) طول ج) به_عدد د) چاپ

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. آسان دو عدد ۲۴ و ۶ را در دو متغیر بگذارید و جمع، تفریق، ضرب و تقسیم آن‌ها را چاپ کنید.
2. آسان مساحت مستطیلی را که طول آن ۱۲ و عرضش ۷ است چاپ کنید.
3. متوسط میانگین سه نمرهٔ ۱۵، ۱۷ و ۲۰ را حساب و چاپ کنید.
4. چالشی طول و عرض مستطیل را از کاربر بگیرید و محیط آن را چاپ کنید. نتیجه را با محاسبهٔ دستی مقایسه کنید.

فصل ۵

شرط و تصمیم‌گیری

در پایان این فصل می‌توانید:

- تصمیم‌گیری در زندگی و برنامه را مقایسه کنید
- از ساختار «اگر ... وگرنه» و تورفتگی درست استفاده کنید
- عدد با عدد و متن با متن را مقایسه کنید
- درست و نادرست را در نتیجهٔ مقایسه بشناسید
- برنامه‌های «بزرگ‌تر از دو عدد»، «قبول/مردود» و «بزرگسال/کودک» بنویسید

۵-۱ تصمیم‌گیری

هر روز تصمیم می‌گیریم: «اگر باران بیاید، چتر برمی‌دارم؛ وگرنه نمی‌برم.» برنامه‌ها هم همین‌طورند. «**شرط**» به برنامه اجازه می‌دهد بسته به یک وضعیت، کار متفاوتی انجام دهد.

فعالیت بدون رایانه

کارت‌های اگر ـ وگرنه. روی کارت‌های رنگی بنویسید: «اگر باران می‌بارد»، «اگر امروز جمعه است»، «اگر نمره‌ام ۲۰ شد». هر کدام را با یک «آن‌وقت ...» و یک «وگرنه ...» کامل کنید و به‌صورت نقش‌آفرینی اجرا کنید: یک نفر کارت را می‌خواند و بقیه با حرکت بدن تصمیم می‌گیرند.

۵-۲ اگر ... وگرنه

condition.t

```
بگذار سن برابر ۲۰
اگر سن بزرگتر یا مساوی ۱۸
    چاپ -بزرگسال-
وگرنه
    چاپ -نابالغ-
```

خروجی

```
بزرگسال
```

چه اتفاقی افتاد؟ رایانه اول شرط «سن بزرگ‌تر یا مساوی ۱۸» را سنجید. چون ۲۰ از ۱۸ کم نیست، شرط **درست** بود و خط‌های زیرِ «اگر» اجرا شد. اگر سن ۱۰ بود، خط‌های زیرِ «وگرنه» اجرا می‌شد.

یادآوری

زیر «اگر» و «وگرنه» باید **تورفتگی** داشته باشید (چهار فاصله). تورفتگی به رایانه نشان می‌دهد کدام خط‌ها «داخلِ» شرط هستند. تورفتگی نادرست، خطا می‌دهد.

۵-۳ مقایسه‌ها

بنویسید	معنی	مثال درست
بزرگتر از	بزرگ‌تر	۵ بزرگتر از ۳
کوچکتر از	کوچک‌تر	۲ کوچکتر از ۴
بزرگتر یا مساوی	بزرگ‌تر یا برابر	۱۸ بزرگتر یا مساوی ۱۸
کوچکتر یا مساوی	کوچک‌تر یا برابر	۳ کوچکتر یا مساوی ۳
==	برابر است با	۳ == ۳
=!	برابر نیست با	۳ =! ۴

نتیجهٔ هر مقایسه یا «درست» است یا «نادرست»:

compare.t

```
چاپ ۳ > ۲
چاپ ۷ =! ۷
چاپ ۴ == ۴
```

خروجی

```
True
False
True
```

نکته

علامت‌های `<` و `>` هم کار می‌کنند. دقت کنید برابری با **دو** مساوی (`==`) نوشته می‌شود؛ یک مساوی برای جای‌گذاری است.

۵-۴ چند شاخه: وگرنه اگر

وقتی بیش از دو حالت داریم، «وگرنه اگر» می‌آوریم. رایانه از بالا شرط‌ها را می‌سنجد و با اولین شرطِ درست کار می‌کند:

grade.t `examples2/conditions/grade.t`

```
بگذار نمره برابر ۱۸
اگر نمره بزرگتر یا مساوی ۱۸
    چاپ -عالی-
وگرنه اگر نمره بزرگتر یا مساوی ۱۲
    چاپ -خوب-
وگرنه
    چاپ -تمرین کن-
```

خروجی

```
عالی
```

pass_fail.t

```
examples2/conditions/pass_fail.t
```

```
بگذار نمره برابر ۱۷
اگر نمره بزرگتر یا مساوی ۱۲
    چاپ -قبول شدی-
وگرنه
    چاپ -نیاز به تمرین بیشتر داری-
```

خروجی

```
قبول شدی
```

۵-۵ ترکیب شرط‌ها: و، یا، نه

- **و**: هر دو شرط باید درست باشند.
- **یا**: یکی از شرط‌ها کافی است.
- **نه**: درست را نادرست و نادرست را درست می‌کند.

logic.t

```
بگذار دما برابر ۲۲
بگذار بارانی برابر نادرست
اگر دما بزرگتر از ۲۰ و نه بارانی
    چاپ -برویم پارک-
وگرنه
    چاپ -در خانه بمانیم-
```

خروجی

```
برویم پارک
```

۵-۶ مقایسهٔ متن

متن‌ها را هم می‌شود با `==` مقایسه کرد؛ ولی متنِ ثابت را نباید مستقیم بعد از `==` نوشت. آن را اول در یک متغیر بگذارید:

password.t

```
بگذار رمز برابر -باز-
چاپ -رمز را بگو:-
بگذار پاسخ برابر بخوان
اگر پاسخ == رمز
    چاپ -در باز شد-
وگرنه
    چاپ -رمز اشتباه است-
```

خروجی

```
ورودی تایپ‌شده: باز
رمز را بگو:
در باز شد
```

۵-۷ برنامه‌هایی با شرط

۱) بزرگ‌تر از دو عدد:

bigger.t

```
بگذار الف برابر ۷
بگذار ب برابر ۱۲
اگر الف بزرگتر از ب
    چاپ -الف بزرگ‌تر است-
وگرنه اگر الف == ب
    چاپ -هر دو برابرند-
وگرنه
    چاپ -ب بزرگ‌تر است-
```

خروجی

```
ب بزرگ‌تر است
```

۲) زوج یا فرد با باقی‌ماندهٔ تقسیم: عددی زوج است که باقی‌ماندهٔ تقسیمش بر ۲ صفر باشد.

even.t

```
بگذار عدد برابر ۹
اگر عدد % ۲ == ۰
    چاپ -زوج-
وگرنه
    چاپ -فرد-
```

خروجی

```
فرد
```

توجه

اگر عددی را از کاربر می‌گیرید، اول با `به_عدد` آن را به عدد تبدیل کنید و در متغیر بریزید. سپس مقایسه کنید. نوشتن `اگر به_عدد با س > ۵` بدون پرانتز خطا می‌دهد.

grade2.t

```
چاپ -نمره‌ات چند شد؟-
بگذار متن برابر بخوان
بگذار نمره برابر به_عدد با متن
اگر نمره بزرگتر یا مساوی ۱۰
    چاپ -قبول-
وگرنه
    چاپ -مردود-
```

خروجی

ورودی تایپ‌شده: ۱۴

نمره‌ات چند شد؟

قبول

مثال‌های آمادهٔ این فصل در گیتهاب

پوشهٔ **examples2/conditions** را در مخزن رسمی ببینید و فایل‌ها را بخوانید و اجرا کنید:

```
conditions/grade.t
conditions/pass_fail.t
```

https://github.com/worker2025/tirotir/tree/main/examples2/conditions

مطالعهٔ آزاد: پایتون

در پایتون به‌جای «اگر / وگرنه اگر / وگرنه» می‌نویسیم `if / elif / else`؛ پس از شرط **دونقطه** می‌گذاریم و بدنه با **تورفتگی** می‌آید. «و، یا، نه» می‌شود `and, or, not`. در پایتون هم باید ورودی را به عدد تبدیل کرد (`int` یا `float`):

```
age = 20
if age >= 18:
    print("adult")
else:
    print("minor")

score = int(input())
if score >= 18:
    print("great")
elif score >= 12:
    print("good")
else:
    print("practice")

print(3 > 2 and 2 > 1, not 3 > 2)
```

خروجی:

```
adult
good
True False
```

مقایسه‌ها در پایتون هم `<= >= < > != ==` هستند و مقدارهای `True` و `False` می‌دهند.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. برای «بزرگ‌تر یا مساوی ۱۸» در تیروتیر چه می‌نویسیم؟

الف) سن بزرگتر یا مساوی ۱۸ ب) سن =< ۱۸ ج) سن بزرگ ۱۸ د) سن مساوی ۱۸ یا بزرگ

۲. خروجی `چاپ ۷ != ۷` چیست؟

الف) True ب) False ج) ۷ د) خطا

۳. چرا خط‌های زیر «اگر» باید تورفته باشند؟

الف) برای زیبایی ب) تا رایانه بداند کدام خط‌ها داخل شرط‌اند ج) تا سرعت بیشتر شود د) لازم نیست

۴. اگر نمره ۱۴ باشد، برنامهٔ `grade.t` (عالی ≥ ۱۸، خوب ≥ ۱۲، وگرنه تمرین کن) چه چاپ می‌کند؟

الف) عالی ب) خوب ج) تمرین کن د) هیچ

۵. کدام عبارت فقط وقتی درست است که **هر دو** شرط درست باشند؟

الف) الف یا ب ب) نه الف ج) الف و ب د) الف == ب

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. **آسان** برنامه‌ای بنویسید که سن را در یک متغیر بگذارد و بنویسد «کودک» (کمتر از ۱۲)، «نوجوان» (۱۲ تا ۱۷) یا «بزرگسال».

2. **آسان** دو عدد را از کاربر بگیرید و بزرگ‌تر را چاپ کنید.

3. **متوسط** دمای هوا را از کاربر بگیرید: بیش از ۳۰ «هوا گرم است»، ۱۵ یا بیشتر «هوا معتدل است»، و در غیر این صورت «هوا سرد است».

4. **چالشی** عدد را از کاربر بگیرید و بگویید زوج است یا فرد، و هم‌چنین آیا بزرگ‌تر از ۱۰۰ هست یا نه.

فصل ۶

حلقه و تکرار

در پایان این فصل می‌توانید:

- توضیح دهید چرا برنامه‌ها از حلقه استفاده می‌کنند
- حلقه‌های «تکرار»، «تاوقتی» و «برای» را بنویسید
- الگوهای متنی و جدول ضرب بسازید
- با حلقه و متغیر بشمارید و جمع بزنید
- حلقه را با شرط ترکیب کنید

۶-۱ چرا حلقه؟

فرض کنید باید یک کلمه را ۱۰۰ بار چاپ کنید. نوشتن ۱۰۰ خط «چاپ» خسته‌کننده است! «**حلقه**» به رایانه می‌گوید یک کار را چند بار تکرار کن.

فعالیت بدون رایانه

الگوی رنگ‌ها. با کارت‌های رنگی یک الگو بچینید: قرمز، آبی، قرمز، آبی، ... . بخش تکرارِ الگو چیست؟ آن را با یک جمله بگویید: «قرمز و آبی را ۴ بار تکرار کن».

۶-۲ تکرار ... بار

loop.t

```
تکرار ۴ بار
    چاپ -سلام-
```

خروجی

```
سلام
سلام
سلام
سلام
```

خط‌های تورفته زیر «تکرار» «بدنهٔ حلقه»اند و هر بار اجرا می‌شوند. حالا یک شمارنده:

repeat_count.t

examples2/loops/repeat_count.t

```
بگذار شمارنده برابر ۰
تکرار ۵ بار
    بگذار شمارنده برابر شمارنده + ۱
    چاپ شمارنده
```

خروجی

```
1
2
3
4
5
```

یادآوری

اگر تعداد تکرار را در یک متغیر گذاشتید، «بار» را ننویسید: `تکرار ن`. (نوشتن `تکرار ن بار` با متغیر، در نسخهٔ ۲ خطا می‌دهد؛ ولی `تکرار ۵ بار` با عدد درست است.)

loop2.t

```
بگذار تعداد برابر ۳
تکرار تعداد
    چاپ -تیروتیر-
```

خروجی

```
تیروتیر
تیروتیر
تیروتیر
```

۳-۶ تاوقتی

حلقهٔ «**تاوقتی**» تا وقتی شرط درست است کار را تکرار می‌کند. مثلاً شمارش از ۱ تا ۳:

while.t

```
بگذار ن برابر ۱
تاوقتی ن =< ۳
    چاپ ن
    بگذار ن برابر ن + ۱
```

خروجی

```
1
2
3
```

توجه

اگر وسط حلقه مقدار را تغییر ندهید (`ن برابر ن + ۱`)، شرط هیچ‌وقت نادرست نمی‌شود و حلقه بی‌پایان است. تیروتیر آن را پس از تعداد زیادی گام متوقف می‌کند:

endless.t

```
بگذار ن برابر ۱
تاوقتی ن < ۰
    بگذار ن برابر ن + ۱
```

خروجی

```
خطا: اجرای برنامه بیش از حد طولانی شد؛ حلقهٔ بی‌نهایت متوقف شد.
```

۶-۴ برای ... در ...

حلقهٔ «**برای**» روی اعضای یک فهرست (یا حرف‌های یک متن) یکی‌یکی می‌گردد:

for_list.t examples2/loops/for_list.t

```
بگذار رنگ‌ها برابر [-قرمز-، -سبز-، -آبی-]
برای رنگ در رنگ‌ها
    چاپ رنگ
```

خروجی

```
قرمز
سبز
آبی
```

۶-۵ بیرون و ادامه

«**بیرون**» حلقه را فوراً تمام می‌کند و «**ادامه**» به سراغ دور بعدی می‌رود:

break.t

```
بگذار مجموع برابر ۰
برای عدد در [۱، ۲، ۳، ۴، ۵، ۶]
    اگر عدد % ۲ == ۰
        ادامه
    اگر عدد > ۴
        بیرون
    بگذار مجموع برابر مجموع + عدد
چاپ مجموع
```

خروجی

```
4
```

عددهای زوج رد می‌شوند (ادامه) و وقتی عدد از ۴ بزرگ‌تر شد، حلقه تمام می‌شود (بیرون)؛ پس فقط ۱ و ۳ جمع می‌شوند.

۶-۶ الگوها و جدول ضرب

ضرب یک متن در یک عدد، آن متن را چند بار تکرار می‌کند؛ با حلقه می‌شود مثلث ستاره‌ای ساخت:

triangle.t

```
بگذار ردیف برابر ۱
تاوقتی ردیف => ۴
  چاپ -*- * ردیف
  بگذار ردیف برابر ردیف + ۱
```

خروجی

```
*
**
***
****
```

جدول ضرب عدد ۵:

times.t

```
بگذار عدد برابر ۵
بگذار ی برابر ۱
تاوقتی ی => ۱۰
  چاپ عدد * ی
  بگذار ی برابر ی + ۱
```

خروجی

```
5
10
15
20
25
30
35
40
45
50
```

مجموع عددهای ۱ تا ۱۰ با یک «شمارنده» و یک «جمع‌کننده»:

sum.t

```
بگذار مجموع برابر ۰
بگذار ن برابر ۱
تاوقتی ن => ۱۰
  بگذار مجموع برابر مجموع + ن
  بگذار ن برابر ن + ۱
چاپ مجموع
```

خروجی

```
55
```

مثال‌های آمادهٔ این فصل در گیتهاب

پوشهٔ **examples2/loops** را در مخزن رسمی ببینید و فایل‌ها را بخوانید و اجرا کنید:

```
loops/repeat_count.t
loops/for_list.t
https://github.com/worker2025/tirotir/tree/main/examples2/loops
```

مطالعهٔ آزاد: پایتون

در پایتون «تکرار n بار» با `for ... in range(n)` و «تاوقتی» با `while` نوشته می‌شود. `range(1, 11)` یعنی از ۱ تا ۱۰. «بیرون» و «ادامه» در پایتون `break` و `continue` هستند:

```
for i in range(4):
    print("hello")

n = 1
while n <= 3:
    print(n)
    n = n + 1

total = 0
for n in range(1, 11):
    total = total + n
print(total)

for row in range(1, 5):
    print("*" * row)

for color in ["red", "green", "blue"]:
    print(color)
```

خروجی:

```
hello
hello
hello
hello
1
2
3
55
*
**
***
****
red
green
blue
```

دقت: `range(4)` عددهای ۰ تا ۳ را می‌دهد؛ یعنی چهار بار تکرار، ولی از صفر شروع می‌شود.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. برنامهٔ `تکرار ۳ بار` با بدنهٔ `-چاپ - الف` چند خط چاپ می‌کند؟

الف) ۱ ب) ۲ ج) ۳ د) ۴

۲. حلقهٔ «تاوقتی» تا کی ادامه دارد؟

الف) تا ۱۰ بار ب) تا وقتی شرطش درست است ج) تا وقتی شرطش نادرست است د) همیشه

۳. حلقهٔ `برای رنگ در رنگ‌ها` چه کار می‌کند؟

الف) رنگ‌ها را حذف می‌کند ب) برای هر عضو فهرست بدنه را یک بار اجرا می‌کند ج) فقط عضو آخر را می‌دهد د) فهرست را مرتب می‌کند

۴. کدام دستور حلقه را فوراً تمام می‌کند؟

الف) ادامه ب) بیرون ج) تکرار د) برای

۵. اگر در حلقهٔ `تا وقتی ن <= ۳` مقدار ن را زیاد نکنیم چه می‌شود؟

الف) حلقه بی‌پایان می‌شود (تیروتیر متوقفش می‌کند) ب) حلقه سه بار اجرا می‌شود ج) برنامه خودکار درست می‌شود د) هیچ اتفاقی نمی‌افتد

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. **آسان** کلمهٔ «سلام» را پنج بار چاپ کنید (با «تکرار»).
2. **آسان** اعداد ۱ تا ۱۰ را با حلقهٔ «تاوقتی» چاپ کنید.
3. **متوسط** جدول ضرب عدد دلخواه ۷ را تا ۱۰ بسازید.
4. **چالشی** مجموع عددهای فردِ ۱ تا ۹ را با حلقه حساب کنید (از «ادامه» کمک بگیرید).

فصل ۷

تابع و استفادهٔ مجدد از کد

در پایان این فصل می‌توانید:

- تابع را «دستور ساختهٔ خودمان» توضیح دهید
- تابعی با ورودی بنویسید و نتیجه را با «بازگردان» بدهید
- تابع را با مقدارهای مختلف فراخوانی کنید
- تابع را در شرط و حلقه به‌کار ببرید
- برنامهٔ بزرگ را به چند تابع کوچک تقسیم کنید

۷-۱ تابع؛ ماشین جادویی

اگر یک کار را چند جا از برنامه لازم داشته باشید، هر بار نوشتنش وقت‌گیر و پر از اشتباه است. «**تابع**» یک دستورِ ساختهٔ خودمان است: یک بار می‌نویسیم، هر جا لازم شد صدایش می‌زنیم.

فعالیت بدون رایانه

ماشین جادویی. یک جعبهٔ مقوایی بردارید که یک سوراخ ورودی و یک سوراخ خروجی دارد. روی آن بنویسید «ماشین دو برابر کن». یک نفر داخل جعبه می‌نشیند. عدد ۳ را روی کاغذ می‌نویسید و از ورودی می‌دهید؛ او ۶ را می‌نویسد و بیرون می‌دهد. ورودی، کار داخل ماشین و خروجی: همان تابع است.

۷-۲ ساختار تابع

add_numbers.t — examples2/functions/add_numbers.t

```
تابع جمع با الف و ب
    بازگردان الف + ب

بگذار نتیجه برابر جمع با ۷ و ۵
چاپ نتیجه
```

خروجی

```
12
```

- `تابع جمع با الف و ب`: تابعی به نام «جمع» که دو ورودی (پارامتر) با نام‌های الف و ب می‌گیرد.
- `بازگردان الف + ب`: نتیجه را به صدازننده برمی‌گرداند.
- `جمع با ۷ و ۵`: تابع را «فراخوانی» می‌کنیم؛ ۷ و ۵ «آرگومان» هستند (چند ورودی را با «و» جدا می‌کنیم).

یک ورودی، یک خروجی:

square.t

```
تابع مربع با عدد
    بازگردان عدد * عدد

چاپ مربع با ۵
چاپ مربع با ۱۲
چاپ مربع با ۰٫۵
```

خروجی

```
25
144
0.25
```

نکته

تابع ساخته‌شده را می‌توان هر چند بار و با ورودی‌های مختلف صدا زد. هر تابع باید **دست‌کم یک ورودی** داشته باشد.

۳-۷ تابع‌های دیگر

examples2/functions/format_result.t — format_result.t

```
تابع پیام با نام
    بازگردان "سلام " + نام

چاپ پیام با "دوست"
```

خروجی

```
سلام دوست
```

examples2/functions/even_check.t — even_check.t

```
تابع زوج_است با عدد
    اگر عدد % ۲ == ۰
        بازگردان درست
    وگرنه
        بازگردان نادرست

چاپ زوج_است با ۸
چاپ زوج_است با ۹
```

خروجی

```
True
False
```

خروجی `True` یعنی «درست» و `False` یعنی «نادرست». تابع می‌تواند شرط داشته باشد و در شرط‌ها هم به‌کار برود:

max.t

```
تابع بزرگتر_از با الف و ب
    اگر الف > ب
        بازگردان الف
    بازگردان ب

چاپ بزرگتر_از با ۳ و ۹
بگذار دو_برابر برابر ۲ * بزرگتر_از با ۳ و ۹
چاپ دو_برابر
```

خروجی

```
9
18
```

۷-۴ تابع + شرط + حلقه

examples2/functions/function_condition_loop.t

function_condition_loop.t

```
تابع جمع_مثبت‌ها با عددها
    بگذار مجموع برابر ۰
    برای عدد در عددها
        اگر عدد بزرگتر از ۰
            بگذار مجموع برابر مجموع + عدد
    بازگردان مجموع

بگذار داده‌ها برابر [۰، ۵، ۲، ۰، ۴]
چاپ جمع_مثبت‌ها با داده‌ها
```

خروجی

```
11
```

این تابع فقط عددهای مثبت فهرست را جمع می‌کند: ۵ + ۲ + ۴ = ۱۱ (صفرها اثری ندارند).

۷-۵ تابع‌های کوچک و گویا

- هر تابع یک کار مشخص انجام دهد.
- نام تابع نشان دهد چه می‌کند: `زوج_است`، `مربع`، `پیام`.
- برنامهٔ بزرگ را به چند تابع کوچک بشکنید و آن‌ها را به‌هم بچسبانید.

rectangle_functions.t

```
تابع مساحت_مستطیل با طول و عرض
    بازگردان طول * عرض

تابع محیط_مستطیل با طول و عرض
    بازگردان ۲ * (طول + عرض)

چاپ مساحت_مستطیل با ۸ و ۵
چاپ محیط_مستطیل با ۸ و ۵
```

خروجی

```
40
26
```

توجه

متغیرهایی که داخل تابع می‌سازید فقط داخل همان تابع شناخته‌اند. بیرون از تابع آن‌ها را نمی‌بینید.

scope.t

```
تابع دو_برابر با عدد
    بگذار نتیجه برابر عدد * ۲
    بازگردان نتیجه

چاپ دو_برابر با ۴
چاپ نتیجه
```

خروجی

```
TIR2001: متغیر «نتیجه» پیدا نشد.
پیشنهاد: ابتدا آن را با «بگذار» تعریف کن.
```

مثال‌های آمادهٔ این فصل در گیتهاب

پوشهٔ **examples2/functions** را در مخزن رسمی ببینید و فایل‌ها را بخوانید و اجرا کنید:

```
functions/add_numbers.t
functions/even_check.t
functions/format_result.t
functions/function_condition_loop.t
```

https://github.com/worker2025/tirotir/tree/main/examples2/functions

مطالعهٔ آزاد: پایتون

در پایتون تابع با `def` ساخته می‌شود، پس از نام و پارامترها دونقطه می‌آید و نتیجه با `return` برگردانده می‌شود. در پایتون تابعِ بدون ورودی هم مجاز است:

```
def add(a, b):
    return a + b

def is_even(number):
    return number % 2 == 0

def hello():
    print("hello")

print(add(7, 5))
print(is_even(8), is_even(9))
hello()
```

خروجی:

```
12
True False
hello
```

تفاوت‌ها: `def` به‌جای «تابع»، پرانتز و ویرگول برای ورودی‌ها (`add(7, 5)`) به‌جای «با ... و ...».

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. کدام دستور خروجی تابع را برمی‌گرداند؟

الف) چاپ ب) بخوان ج) بازگردان د) تکرار

۲. برنامهٔ زیر چه چاپ می‌کند؟

`تابع مربع با عدد` / `بازگردان عدد * عدد` / `چاپ مربع با ۵`

الف) ۱۰ ب) ۲۵ ج) ۵ د) خطا

۳. تابع `جمع با الف و ب` چند ورودی دارد؟

الف) ۱ ب) ۲ ج) ۳ د) هیچ

۴. یکی از مزیت‌های تابع چیست؟

الف) برنامه کندتر می‌شود ب) از تکرار کد جلوگیری می‌کند ج) خطا را بیشتر می‌کند د) حافظه را پاک می‌کند

۵. اگر داخل تابع متغیری بسازیم، آن را بیرون از تابع...

الف) می‌بینیم ب) نمی‌بینیم ج) فقط در حلقه می‌بینیم د) همیشه صفر است

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. آسان تابعی بنویسید که یک عدد بگیرد و دو برابر آن را بدهد؛ سپس آن را برای ۴ چاپ کنید.
2. آسان تابع «مکعب» بنویسید و آن را با ۳ صدا بزنید.
3. متوسط تابعی بنویسید که یک نام بگیرد و «سلام نام» را برگرداند؛ آن را سه بار با سه نام مختلف صدا بزنید.
4. چالشی تابعی بنویسید که یک عدد بگیرد و اگر بزرگ‌تر از ۱۰ بود «بزرگ» و وگرنه «کوچک» برگرداند. سپس برای فهرست [۵، ۱۵، ۱۰] هر عضو را بررسی و چاپ کنید.

فصل ۸

فهرست و فرهنگ

در پایان این فصل می‌توانید:

- نیاز به نگه‌داشتن چند مقدار با هم را توضیح دهید
- فهرست بسازید، عضوها را بخوانید و عضو اضافه کنید
- فهرست را با حلقه پیمایش کنید
- فرهنگ (کلید و مقدار) بسازید و مقدار یک کلید را بخوانید
- برنامه‌های «فهرست خرید» و «دفترچهٔ تلفن» بنویسید

۸-۱ فهرست؛ مجموعهٔ مرتب

برای نمرهٔ ده درس، ده متغیر نمی‌سازیم! «**فهرست**» چند مقدار را به ترتیب با هم نگه می‌دارد. اعضا داخل کروشه `[ ]` و با «،» (یا ویرگول) جدا می‌شوند. هر عضو یک **شمارهٔ جایگاه** دارد که از **صفر** شروع می‌شود:

create_and_read.t — examples2/lists/create_and_read.t

```
بگذار میوه‌ها برابر [-سیب-، -موز-، -پرتقال-]
چاپ میوه‌ها[۰]
چاپ میوه‌ها[۲]
```

خروجی

```
سیب
پرتقال
```

فعالیت بدون رایانه

کارت‌های شماره‌دار. پنج کارت بردارید و روی هر کدام یک میوه بنویسید. آن‌ها را در یک ردیف بچینید و زیرشان به‌ترتیب ۰، ۱، ۲، ۳، ۴ بنویسید. حالا از دوستتان بپرسید: «کارت شمارهٔ ۲ چیست؟» به همین دلیل میوه‌ها[۲] سومین میوه است.

یادآوری

شمارهٔ جایگاه از **صفر** شروع می‌شود. عضو اول: `[۰]` . اگر شمارهٔ بیرون از فهرست بدهید، تیروتیر پیام «دسترسی به فهرست یا فرهنگ نامعتبر است» می‌دهد.

۸-۲ افزودن و تعداد

add_item.t examples2/lists/add_item.t

```
بگذار عددها برابر [۱، ۲، ۳]
افزودن با عددها و ۴
چاپ عددها
چاپ طول با عددها
```

خروجی

```
[1, 2, 3, 4]
4
```

`افزودن با فهرست و مقدار` عضو تازه به آخر فهرست می‌افزاید و `طول با فهرست` تعداد اعضا را می‌دهد. با `حذف با فهرست و جایگاه` می‌شود عضوی را برداشت. تغییر یک عضو هم با جایگاهش ممکن است:

change_list.t

```
بگذار ل برابر [-الف-، -ب-، -ج-]
بگذار ز برابر -ز-
ل[۱] = ز
چاپ ل
چاپ حذف با ل و ۰
چاپ ل
```

خروجی

```
['الف', 'ز', 'ج']

الف

['ز', 'ج']
```

۸-۳ پیمایش فهرست

avg_list.t

```
بگذار نمره‌ها برابر [۱۸، ۱۶، ۲۰، ۱۹]
بگذار مجموع برابر ۰
برای نمره در نمره‌ها
  بگذار مجموع برابر مجموع + نمره
بگذار میانگین برابر مجموع / طول با نمره‌ها
چاپ میانگین
```

خروجی

```
18.25
```

و همین برنامه به‌صورت کامل در مخزن رسمی هم هست:

00_رسمی.t

```
examples2/00_رسمی.t
```

```
بگذار نام برابر -سارا-
بگذار نمره‌ها برابر [۱۸، ۱۶، ۲۰، ۱۹]
بگذار مجموع برابر ۰

برای نمره در نمره‌ها
    بگذار مجموع برابر مجموع + نمره

بگذار میانگین برابر مجموع / ۴

اگر میانگین بزرگتر یا مساوی ۱۸
    چاپ -عالی است-
وگرنه اگر میانگین بزرگتر یا مساوی ۱۲
    چاپ -قبول شده است-
وگرنه
    چاپ -باید بیشتر تلاش کند-
```

خروجی

```
عالی است
```

۸-۴ فهرست خرید

shopping.t

```
بگذار خرید برابر [-نان-، -شیر-]
افزودن با خرید و -تخم‌مرغ-
چاپ -تعداد اقلام:-
چاپ طول با خرید
برای قلم در خرید
    چاپ قلم
```

خروجی

```
تعداد اقلام:
3
نان
شیر
تخم‌مرغ
```

۸-۵ فرهنگ؛ کلید و مقدار

در یک «**فرهنگ**» هر مقدار یک **کلید** (برچسب) دارد؛ مثل دفترچهٔ تلفن (نام ← شماره) یا فرهنگ لغت (واژه ← معنی). برخلاف فهرست، به‌جای شمارهٔ جایگاه، با کلید سراغ مقدار می‌رویم:

person_record.t — examples2/dictionaries/person_record.t

```
بگذار شخص برابر فرهنگ
    نام برابر -نرگس-
    شهر برابر -شیراز-
    سن برابر ۲۱
چاپ شخص[نام]
چاپ شخص[شهر]
چاپ شخص[سن]
```

خروجی

```
نرگس
شیراز
21
```

ساختار فرهنگ: `بگذار نام برابر فرهنگ` و در خط‌های تورفتهٔ بعد، هر عضو به شکل `کلید برابر مقدار`. برای خواندن: `شخص[نام]`.

> **نکته**
>
> اگر نام کلید با نام یک متغیرِ موجود یکی باشد، تیروتیر مقدار همان متغیر را به‌جای خودِ کلید به‌کار می‌برد. بهتر است نام متغیرها و کلیدها یکی نباشد.

phonebook.t

```
بگذار دفترچه برابر فرهنگ
    علی برابر -۰۹۱۲۱۱۱-
    سارا برابر -۰۹۳۵۲۲۲-
بگذار کلید برابر بخوان
چاپ دفترچه[کلید]
بگذار شمارهٔ_پویا برابر -۰۹۱۹۳۳۳-
دفترچه[-پویا-] = شمارهٔ_پویا
چاپ دفترچه[-پویا-]
چاپ طول با دفترچه
```

خروجی

```
ورودی تایپ‌شده: سارا
۰۹۳۵۲۲۲
۰۹۱۹۳۳۳
3
```

برای اضافه‌کردن عضو تازه به فرهنگ، کلید را به‌صورت متن بنویسید (`دفترچه[-پویا-] = ...`). اگر مقدار، متن است، آن را اول در یک متغیر بگذارید (چون بعد از `=` متنِ ثابت پذیرفته نمی‌شود).

نکته

در نسخهٔ ۲ حلقهٔ «برای» فقط روی **فهرست** و **متن** می‌چرخد، نه روی فرهنگ. اگر لازم دارید روی چند کلید بگردید، کلیدها را در یک فهرست بنویسید و با آن‌ها از فرهنگ بخوانید.

dict_keys.t

```
بگذار میوه برابر فرهنگ
    سیب برابر ۵
    موز برابر ۳
بگذار کلیدها برابر [-سیب-، -موز-]
برای کلید در کلیدها
    چاپ میوه[کلید]
```

خروجی

```
5
3
```

۸-۶ فهرست یا فرهنگ؟

	فهرست	فرهنگ
دسترسی با	شمارهٔ جایگاه (۰، ۱، ۲ ...)	کلید (نام، شهر ...)
مناسب برای	چند مقدار هم‌نوع و مرتب (نمره‌ها، اقلام خرید)	مشخصات یک چیز یا جدول «برچسب ← مقدار»
نمونه	`[۱، ۲، ۳]`	نام برابر -سارا-

مثال‌های آمادهٔ این فصل در گیتهاب

این فایل‌ها در پوشهٔ **examples2** مخزن رسمی هستند؛ بخوانید و اجرا کنید:

```
lists/create_and_read.t
lists/add_item.t
dictionaries/person_record.t
```

https://github.com/worker2025/tirotir/tree/main/examples2

مطالعهٔ آزاد: پایتون

در پایتون فهرست `list` و فرهنگ `dict` نام دارد. فهرست با `[ ]` و فرهنگ با `{ }` نوشته می‌شود. عضو اضافه‌کردن: `append`؛ تعداد: `len`. جایگاه‌ها از صفر شروع می‌شوند، درست مثل تیروتیر:

```
fruits = ["apple", "banana", "orange"]
print(fruits[0])
print(fruits[2])
numbers = [1, 2, 3]
numbers.append(4)
print(numbers)
print(len(numbers))

person = {"name": "Narges", "city": "Shiraz", "age": 21}
print(person["name"])
person["job"] = "student"
print(person)
for key in person:
    print(key)
```

خروجی:

```
apple
orange
[1, 2, 3, 4]
4
Narges
{'name': 'Narges', 'city': 'Shiraz', 'age': 21, 'job': 'student'}
name
city
age
job
```

همین «چاپ فهرست» در تیروتیر هم `[1, 2, 3, 4]` نشان می‌دهد؛ چون مفسر تیروتیر از فهرست و فرهنگ پایتون استفاده می‌کند.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. اولین عضو فهرست با کدام شمارهٔ جایگاه خوانده می‌شود؟

الف) ۱ ب) ۰ ج) -۱ د) ۲

۲. `چاپ طول با [۴، ۵، ۶]` چه چاپ می‌کند؟

الف) ۳ ب) ۴ ج) ۶ د) ۱۵

۳. برای افزودن عضو تازه به انتهای فهرست از چه دستوری استفاده می‌کنیم؟

الف) افزودن با ب) حذف با ج) طول با د) برای

۴. در یک فرهنگ مقدار را با چه چیزی پیدا می‌کنیم؟

الف) شمارهٔ جایگاه ب) کلید ج) طول د) حلقهٔ تکرار

۵. برای نگه‌داشتن «نام، شهر و سن» یک نفر کدام ساختار مناسب‌تر است؟

الف) فهرست اعداد ب) فرهنگ ج) فقط یک متغیر عددی د) حلقه

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. **آسان** فهرستی از سه رنگ بسازید و عضو اول و آخر آن را چاپ کنید.
2. **آسان** مجموع اعداد فهرست [۲، ۴، ۶، ۸] را با حلقه حساب کنید.
3. **متوسط** فهرست دو دوست بسازید، نام دوست دیگری را اضافه کنید و فهرست و تعدادشان را چاپ کنید.
4. **چالشی** یک فرهنگ کوچک «واژه ← معنی» با سه واژه (مثلاً کتاب، قلم، میز ← انگلیسی‌شان) بسازید و معنی واژه‌ای را که کاربر می‌نویسد نشان دهید.

فصل ۹

گرافیک و نقاشی با برنامه

در پایان این فصل می‌توانید:

- مفهوم صحنه، مختصات و قلم را توضیح دهید
- رنگ و ضخامت قلم را تنظیم کنید
- شکل‌های پایه را رسم کنید و اندازهٔ آن‌ها را تغییر دهید
- با حلقه و شکل، طرح‌های تکراری بسازید
- تصویر را زنده در VS Code ببینید و به‌صورت SVG ذخیره کنید

۹-۱ قلمِ لاک‌پشتی

در تیروتیر نقاشی با یک «**قلم**» انجام می‌شود که روی صفحه راه می‌رود و هر جا برود خط می‌کشد؛ درست مثل لاک‌پشتی که راهش را با رنگ علامت می‌زند. قلم از **وسط صفحه** شروع می‌کند و رو به **راست** است. دستورهای اصلی:

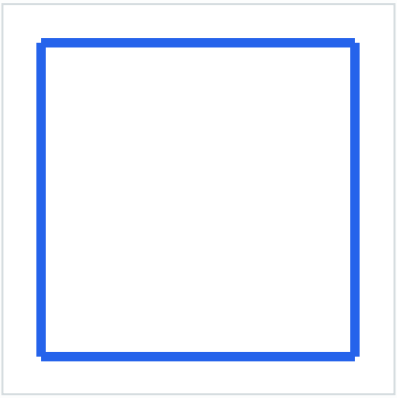
دستور	کار
پنجره بازکن با ۸۰۰ و ۶۰۰	صحنه‌ای به عرض ۸۰۰ و ارتفاع ۶۰۰ می‌سازد
رنگ قلم برابر -آبی-	رنگ خط را تعیین می‌کند
ضخامت قلم برابر ۳	کلفتی خط را تعیین می‌کند
حرکت با ۱۰۰	قلم ۱۰۰ واحد جلو می‌رود و خط می‌کشد
عقب برو با ۵۰	قلم ۵۰ واحد عقب می‌رود
چرخش با ۹۰	قلم ۹۰ درجه (خلاف جهت ساعت) می‌چرخد
قلم بالا / قلم پایین	قلم را از صفحه بلند می‌کند / می‌گذارد
مربع با ۱۲۰ ، دایره با ۸۰ ، مثلث با ۱۲۰ ، مستطیل با ۱۸۰ و ۹۰	شکل‌های آماده
پاک کن	همهٔ خط‌های کشیده‌شده را پاک می‌کند

نکته

رنگ‌هایی که فارسی نوشته می‌شوند: سیاه، سفید، قرمز، سبز، آبی، زرد، نارنجی، بنفش. نام انگلیسی رنگ (مثل `pink`) یا کد رنگ (مثل `#16a34a`) هم کار می‌کند. در نسخهٔ ۲ شکل‌ها **فقط خط‌کشی** می‌شوند و رنگِ داخل ندارند، و هنوز انیمیشن نیست.

۹-۲ رسم شکل

hello_shape.t — examples2/graphics/hello_shape.t

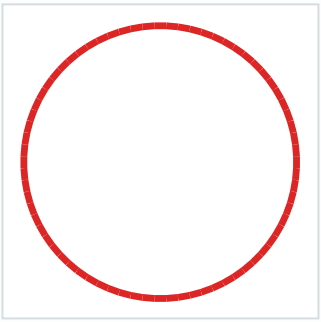


مربع با قلم آبی

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -آبی-
ضخامت قلم برابر ۳
حرکت با ۱۰۰
چرخش با ۹۰
حرکت با ۱۰۰
چرخش با ۹۰
حرکت با ۱۰۰
چرخش با ۹۰
حرکت با ۱۰۰
```

قلم چهار بار «حرکت ۱۰۰» و «چرخش ۹۰» انجام داد و مربع کشیده شد. این کار را دستور آمادهٔ `مربع با ۱۰۰` هم انجام می‌دهد:

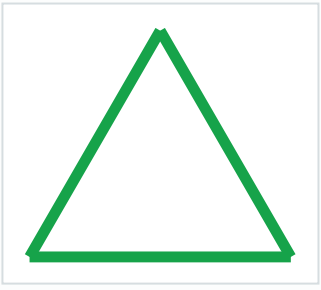
circle.t — examples2/graphics/circle.t



دایره

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -قرمز-
ضخامت قلم برابر ۴
دایره با ۸۰
```

triangle.t — examples2/graphics/triangle.t



مثلث

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -سبز-
ضخامت قلم برابر ۵
مثلث با ۱۲۰
```

rectangle.t — examples2/graphics/rectangle.t



مستطیل

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -بنفش-
ضخامت قلم برابر ۳
مستطیل با ۱۸۰ و ۹۰
```

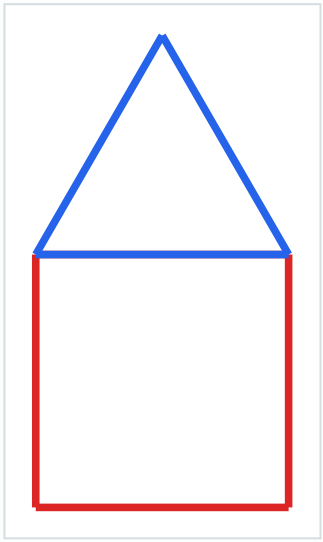
یادآوری

«مختصات» یعنی جای هر نقطه روی صفحه. قلم از نقطهٔ وسط شروع می‌کند و هر «حرکت» او را در جهتِ فعلی جابه‌جا می‌کند. «چرخش» فقط جهت را عوض می‌کند و خطی نمی‌کشد.

۹-۳ ترکیب شکل‌ها: یک خانه

house.t

پنجره بازکن با ۸۰۰ **و** ۶۰۰
رنگ قلم برابر -قرمز-
ضخامت قلم برابر ۳
مربع با ۱۰۰
قلم بالا
چرخش با ۹۰
حرکت با ۱۰۰
چرخش با ۲۷۰
قلم پایین
رنگ قلم برابر -آبی-
مثلث با ۱۰۰



خانه با دیوار قرمز و سقف آبی

پس از رسم دیوار، قلم را بلند کردیم (`قلم بالا`)، به گوشهٔ بالای دیوار بردیم، دوباره گذاشتیم و سقف را کشیدیم.

۹-۴ حلقه و گرافیک

با حلقه می‌شود با چند خط، طرح‌های زیبا ساخت. ستارهٔ پنج‌پر:

star.t

پنجره بازکن با ۸۰۰ **و** ۶۰۰
رنگ قلم برابر -نارنجی-
ضخامت قلم برابر ۳
تکرار ۵ **بار**
حرکت با ۱۰۰
چرخش با ۱۴۴

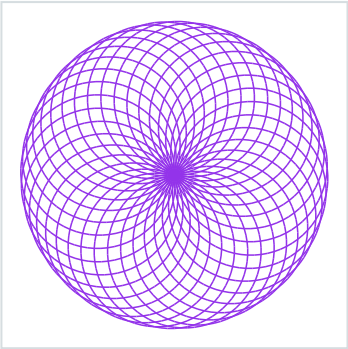


ستارهٔ پنج‌پر

گل با ۳۶ دایره‌ای که کمی می‌چرخند:

flower.t

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -بنفش-
ضخامت قلم برابر ۱
تکرار ۳۶ بار
  دایره با ۵۰
  چرخش با ۱۰
```

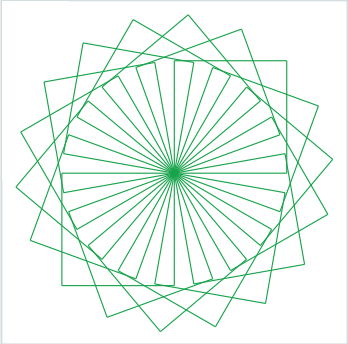


طرح گل

و مربع‌های چرخان:

squares.t

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -سبز-
ضخامت قلم برابر ۱
تکرار ۱۸ بار
  مربع با ۱۰۰
  چرخش با ۲۰
```

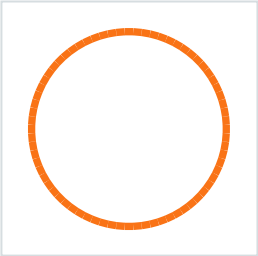


مربع‌های چرخان

قلم با «پاک کن» صفحه را پاک می‌کند؛ نمونهٔ رسمی مخزن، اول یک مربع می‌کشد و بعد پاک می‌کند و یک دایره می‌کشد، پس در پایان فقط دایره می‌ماند:

examples2/graphics/multiple_shapes.t — multiple_shapes.t

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -آبی-
ضخامت قلم برابر ۳
  مربع با ۸۰
  پاک کن
  رنگ قلم برابر -نارنجی-
  دایره با ۴۰
```



فقط دایره باقی می‌ماند

۵-۹ دیدن و ذخیرهٔ نتیجه

- در VS Code: راست‌کلیک ← «TirotirLang: نمایش Graphics زنده»؛ تصویر بدون ساخت فایل کنار برنامه دیده می‌شود.
- در خط فرمان تصویر را به‌صورت فایل **SVG** ذخیره می‌کنیم؛ این فایل را با هر مرورگر می‌شود باز کرد.

خط فرمان

```
tirotir graphics star.t --scene star.svg
```

توجه

دستور ساده‌ی `tirotir run star.t` برای برنامهٔ گرافیکی خطای «قابلیت graphics فعال نیست» می‌دهد؛ چون قابلیت‌ها (Capability) به‌طور پیش‌فرض خاموش‌اند. از `tirotir graphics` استفاده کنید یا (فصل ۱۴) قابلیت را روشن کنید: `tirotir run star.t --capability graphics`.

ایمنی و سلامت

هنگام کار با تصویر، فاصلهٔ چشم تا نمایشگر را رعایت کنید و پیش از هر چاپ، نتیجه را روی نمایشگر بررسی کنید تا کاغذ هدر نرود.

مثال‌های آمادهٔ این فصل در گیتهاب

پوشهٔ **examples2/graphics** را در مخزن رسمی ببینید و فایل‌ها را بخوانید و اجرا کنید:

```
graphics/hello_shape.t
graphics/circle.t
graphics/triangle.t
graphics/rectangle.t
graphics/multiple_shapes.t
```

https://github.com/worker2025/tirotir/tree/main/examples2/graphics

مطالعهٔ آزاد: پایتون

پایتون ماژول آمادهٔ `turtle` را دارد که دقیقاً همین قلم لاک‌پشتی است: `forward` همان «حرکت»، `left` همان «چرخش» (خلاف جهت ساعت) و `color` و `pensize` رنگ و ضخامت‌اند.

```
import turtle

pen = turtle.Turtle()
pen.color("blue")
pen.pensize(3)
for _ in range(4):
    pen.forward(100)
    pen.left(90)

# ستاره
pen.color("orange")
for _ in range(5):
    pen.forward(100)
    pen.left(144)

turtle.done()
```

این برنامه به صفحه‌نمایش یا دستگاه صوتی نیاز دارد؛ روی رایانهٔ خودتان اجرا کنید.

این برنامه پنجرهٔ گرافیکی باز می‌کند و باید روی رایانهٔ خودتان اجرا شود.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. قلم تیروتیر در ابتدا کجاست و رو به کدام سمت؟

الف) بالای صفحه، رو به پایین ب) وسط صفحه، رو به راست ج) گوشهٔ چپ، رو به بالا د) بیرون از صفحه

۲. برای رسم مربع با «حرکت» و «چرخش»، قلم هر بار چند درجه می‌چرخد؟

الف) ۴۵ ب) ۶۰ ج) ۹۰ د) ۱۸۰

۳. با کدام دستور رنگ خط را آبی می‌کنیم؟

الف) رنگ قلم برابر -آبی- ب) ضخامت قلم برابر آبی ج) حرکت با -آبی- د) چاپ -آبی-

۴. ستارهٔ پنج‌پر با «تکرار ۵ بار» و حرکت ۱۰۰، چند درجه چرخش می‌خواهد؟

الف) ۹۰ ب) ۱۲۰ ج) ۱۴۴ د) ۷۲

۵. تصویر گرافیکی تیروتیر در چه قالبی ذخیره می‌شود؟

الف) MP3 ب) SVG ج) EXE د) DOCX

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. **آسان** یک مربع قرمز با ضلع ۱۲۰ بکشید.
2. **آسان** تصویری با دست‌کم سه شکل و دو رنگ بکشید (مثلاً دایره، مربع و مثلث).
3. **متوسط** با یک حلقه، ستارهٔ پنج‌پر را بکشید و رنگ و ضخامت دلخواه بدهید.
4. **چالشی** طرحی بسازید که ۱۲ مثلثِ چرخان داشته باشد (هر بار ۳۰ درجه بچرخید).

فصل ۱۰

تولید صدا و موسیقی ساده

در پایان این فصل می‌توانید:

- صدا، بسامد (فرکانس) و مدت را به زبان ساده بگویید
- با «نت بنواز» یک نت و سپس گام بسازید
- با حلقه و تابع قطعهٔ کوتاه بسازید
- فایل صوتی WAV تولید و پخش کنید

۱۰-۱ صدا چیست؟

صدا لرزش هواست. هر چه لرزش سریع‌تر باشد صدا «زیرتر» است. تعداد لرزش در ثانیه را «**بسامد**» (فرکانس) می‌نامند. نت «دو» حدود ۲۶۲ لرزش در ثانیه دارد و نت «سی» حدود ۴۹۴. هر نت علاوه بر بسامد یک «**مدت**» هم دارد؛ یعنی چند ثانیه نواخته شود.

فعالیت بدون رایانه

صدای من. یک خط‌کش پلاستیکی را لبهٔ میز بگذارید و با دست بخشی از آن را بگیرید و رها کنید. هر چه بخشِ آزاد کوتاه‌تر باشد، لرزش سریع‌تر و صدا زیرتر است. می‌توانید بخوانید «دو، ر، می، فا، سل، لا، سی» و حس کنید هر نت از قبلی زیرتر است.

۱۰-۲ نواختن یک نت

دستور: `نت بنواز با نام‌نت و مدت`. نام‌نت‌ها: **دو، ر، می، فا، سل، لا، سی**. به‌جای نام، می‌شود بسامد را هم نوشت (مثل ۲۶۲). مدت بر حسب ثانیه است.

نت	دو	ر	می	فا	سل	لا	سی
بسامد تقریبی (لرزش در ثانیه)	۲۶۲	۲۹۴	۳۳۰	۳۴۹	۳۹۲	۴۴۰	۴۹۴

render_only.t — examples2/audio/render_only.t

```
# این نمونه عمداً فقط فایل tirotir-note.wav را تولید می‌کند.
# برای پخش، آن را با tirotir audio اجرا کنید.
نت بنواز با ۲۶۲ و ۰٫۲۰
نت بنواز با ۳۳۰ و ۰٫۲۰
نت بنواز با ۳۹۲ و ۰٫۲۰
```

نتیجه

```
فایل صوتی tirotir-note.wav ساخته شد.
```

برنامه فقط فایل صوتی `tirotir-note.wav` را کنار برنامه می‌سازد. برای پخش:

- در VS Code: راست‌کلیک ← «TirotirLang: پخش Audio»
- در خط فرمان: `tirotir audio scale.t --play-audio`

نکته

ساختن فایل WAV روی هر سیستم‌عاملی کار می‌کند؛ ولی **پخش** صدا به دستگاه صوتی و سامانهٔ صدای رایانهٔ شما وابسته است. اگر پخش نشد، فایل `tirotir-note.wav` را با پخش‌کنندهٔ سیستم باز کنید.

۱۰-۳ گام موسیقی

scale.t — examples2/audio/scale.t

```
# نمونهٔ رسمی: تولید و پخش ترتیبی گام صوتی
نت بنواز با -دو- و ۰٫۱۵
نت بنواز با -ر- و ۰٫۱۵
نت بنواز با -می- و ۰٫۱۵
نت بنواز با -فا- و ۰٫۱۵
نت بنواز با -سل- و ۰٫۱۵
نت بنواز با -لا- و ۰٫۱۵
نت بنواز با -سی- و ۰٫۱۵
```

نتیجه

```
فایل صوتی tirotir-note.wav ساخته شد.
```

هفت نتِ پشت‌سر هم، هر کدام ۰٫۱۵ ثانیه؛ همهٔ نت‌ها در یک فایل WAV به‌ترتیب قرار می‌گیرند.

۱۰-۴ حلقه و تابع در موسیقی

تابعی که یک نت بنوازد و حلقه‌ای که ریتم را تکرار کند:

motif.t

```
تابع بنواز_کوتاه با نام_نت
    نت بنواز با نام_نت و ۰٫۱۵

بنواز_کوتاه با -دو-
بنواز_کوتاه با -می-
بنواز_کوتاه با -سل-
تکرار ۳ بار
    نت بنواز با -دو- و ۰٫۱
```

نتیجه

```
فایل صوتی tirotir-note.wav ساخته شد.
```

و یک ملودی کوتاه و آشنا («ستارهٔ کوچک»):

twinkle.t

```
بگذار نت‌ها برابر [-دو-، -دو-، -سل-، -سل-، -لا-، -لا-، -سل-]
برای نام_نت در نت‌ها
  نت بنواز با نام_نت و ۰٫۲۵
```

نتیجه

فایل صوتی tirotir-note.wav ساخته شد.

ریتم

rhythm_exercise.t — examples2/audio/rhythm_exercise.t

```
# تمرین کوتاه با تکرار نت‌ها original
نت بنواز با -دو- و ۰٫۰۸
نت بنواز با -دو- و ۰٫۰۸
نت بنواز با -سل- و ۰٫۱۶
نت بنواز با -دو- و ۰٫۰۸
نت بنواز با -دو- و ۰٫۰۸
نت بنواز با -سل- و ۰٫۱۶
```

نتیجه

فایل صوتی tirotir-note.wav ساخته شد.

ایمنی و سلامت

صدا را ملایم نگه دارید و ترجیحاً با هدفون گوش دهید؛ از گوش‌دادن طولانی‌مدت با صدای بلند بپرهیزید. به آرامش دیگران در کارگاه احترام بگذارید و پس از پایان کار، هدفون و بلندگو را خاموش کنید.

مثال‌های آمادهٔ این فصل در گیتهاب

پوشهٔ **examples2/audio** را در مخزن رسمی ببینید و فایل‌ها را بخوانید و اجرا کنید:

```
audio/scale.t
audio/melody.t
audio/rhythm_exercise.t
audio/ascending_exercise.t
audio/up_down_exercise.t
audio/render_only.t
https://github.com/worker2025/tirotir/tree/main/examples2/audio
```

مطالعهٔ آزاد: پایتون

پایتون ماژول `wave` دارد که فایل WAV می‌سازد. برای هر نت، موج سینوسی با بسامد آن را می‌نویسیم. این برنامه همان گام را در `scale.wav` می‌سازد و مدت فایل را چاپ می‌کند:

```
import wave, struct, math

rate = 22050
notes = [262, 294, 330, 349, 392, 440, 494]
frames = bytearray()
for freq in notes:
    for i in range(int(rate * 0.15)):
        value = int(12000 * math.sin(2 * math.pi * freq * i / rate))
        frames += struct.pack("<h", value)

with wave.open("scale.wav", "wb") as f:
    f.setnchannels(1)
    f.setsampwidth(2)
    f.setframerate(rate)
    f.writeframes(bytes(frames))

print(len(frames) // 2 / rate, "seconds")
```

خروجی:

```
1.0498412698412698 seconds
```

این همان کاری است که پشت صحنهٔ «نت بنواز» اتفاق می‌افتد: بسامد، مدت و موج صوتی.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. بسامد (فرکانس) چیست؟

الف) بلندی صدا　ب) تعداد لرزش در ثانیه　ج) طول فایل　د) رنگ صدا

۲. کدام یک نام نت پذیرفته‌شده در «نت بنواز» نیست؟

الف) دو　ب) سل　ج) سی　د) هفت

۳. در `نت بنواز با -دو- و ۰٫۲` عدد ۰٫۲ چیست؟

الف) بسامد　ب) مدت (ثانیه)　ج) بلندی صدا　د) شمارهٔ نت

۴. فایلی که برنامهٔ صوتی تولید می‌کند چه نام دارد؟

الف) music.mp3　ب) tirotir-note.wav　ج) sound.exe　د) audio.svg

۵. برای تنظیم سلامت گوش چه باید کرد؟

الف) صدای بلند و هدفون　ب) صدای ملایم و ترجیحاً هدفون　ج) بدون وقفه گوش دادن　د) بلندگو را کنار گوش گذاشتن

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. آسان گام «دو تا سی» را با مدت ۰٫۲ ثانیه برای هر نت بنویسید.
2. متوسط قطعه‌ای ۸نتی بسازید و پخش کنید (با «برای» و یک فهرست از نت‌ها).
3. متوسط با تابع، دو نت «دو» و «سل» را بسازید که پشت‌سر هم تکرار شوند.
4. چالشی الگوی «دو دو سل» را با یک حلقه ۳ بار تکرار کنید.

فصل ۱۱

ساخت برنامهٔ دارای پنجره (رابط گرافیکی)

در پایان این فصل می‌توانید:

- اجزای رابط کاربری را نام ببرید
- پنجره، برچسب، دکمه و کادر ورودی بسازید
- برنامه را با «اجرای GUI» یا `tirotir gui` اجرا کنید
- کارت معرفی و کارت تبریک بسازید
- اصول ساده‌ی طراحی (خوانایی و فاصله‌گذاری) را رعایت کنید

۱۱-۱ رابط کاربری چیست؟

تا اینجا برنامه‌ها در یک صفحهٔ متنی کار می‌کردند. برنامه‌هایی که هر روز می‌بینید، «پنجره» دارند. به‌طور کلی «**رابط کاربری**» (GUI) همان چیزی است که کاربر می‌بیند و با آن کار می‌کند:

- **پنجره:** قاب اصلی برنامه با یک عنوان.
- **برچسب:** متنی که فقط نمایش داده می‌شود.
- **دکمه:** چیزی که کلیک می‌کنیم.
- **ورودی:** کادری که کاربر در آن تایپ می‌کند.

فعالیت بدون رایانه

طراحی روی کاغذ. روی کاغذ A4 یک مستطیل بکشید (پنجره). بالای آن عنوان بنویسید. بعد با مداد رنگی برچسب، دکمه و کادر ورودیِ برنامه‌ای مثل «کارت معرفی» را بکشید. این «طرح اولیه» است. پس از ساخت برنامه، آن را با نتیجهٔ نهایی مقایسه و تفاوت‌ها را بیان کنید.

۱۱-۲ نخستین پنجره

hello_window.t — examples2/gui/hello_window.t

```
پنجره بساز با -سلام تیروتیر- و ۵۰۰ و ۳۰۰
برچسب بساز با -به تیروتیر خوش آمدید-
دکمه بساز با -دکمهٔ نمونه-
ورودی بساز
نمایش پنجره
```

```
پیام در رایانهٔ بدون صفحه‌نمایش گرافیکی
خطا: TIR4001: رابط گرافیکی در این محیط در دسترس نیست؛ Tkinter و desktop لازم است.
```

ترتیب دستورها مهم است: اول پنجره را می‌سازیم؛ بعد اجزا یکی‌یکی از بالا به پایین اضافه می‌شوند؛ و در آخر «نمایش پنجره» آن را روی صفحه می‌آورد. برای اجرا:

- در VS Code: راست‌کلیک ← «TirotirLang: اجرای GUI»
- در خط فرمان: `tirotir gui hello_window.t`

> **توجه**
>
> برنامه‌های GUI فقط روی رایانه‌ای که **نمایشگر واقعی** و کتابخانهٔ **Tkinter** دارد اجرا می‌شوند. در محیط بدون صفحه‌نمایش، پیامی مانند خروجی بالا می‌بینید (TIR4001) که خطای شما نیست. در این حالت می‌توانید با `tirotir check` فقط درستی نوشتن برنامه را بررسی کنید.

hello_window.t — examples2/gui/hello_window.t

```
پنجره بساز با -سلام تیروتیر- و ۵۰۰ و ۳۰۰
برچسب بساز با -به تیروتیر خوش آمدید-
دکمه بساز با -دکمهٔ نمونه-
ورودی بساز
نمایش پنجره
```

نتیجهٔ بررسی

```
برنامه صحیح است.
```

۳-۱۱ کارت معرفی و کارت تبریک

با همان چهار جزء می‌شود برنامه‌های مفید ساخت. «کارت تبریک»:

birthday.t

```
پنجره بساز با -کارت تبریک- و ۴۰۰ و ۲۴۰
برچسب بساز با -تولدت مبارک!-
برچسب بساز با -آرزوی شادی و سلامتی برای تو-
دکمه بساز با -ممنون-
نمایش پنجره
```

نتیجهٔ بررسی

```
برنامه صحیح است.
```

«کارت معرفی» که نام و شهر می‌گیرد:

profile_card.t

```
پنجره بساز با -کارت معرفی- و ۴۰۰ و ۳۰۰
برچسب بساز با -نام شما-
ورودی بساز
برچسب بساز با -شهر شما-
ورودی بساز
دکمه بساز با -ثبت-
نمایش پنجره
```

نتیجهٔ بررسی

برنامه صحیح است.

نمونه‌های رسمی مخزن:

login_form.t — examples2/gui/login_form.t

```
پنجره بساز با -فرم ورود- و ۴۰۰ و ۲۸۰
برچسب بساز با -نام کاربری-
ورودی بساز
برچسب بساز با -گذرواژه-
ورودی بساز
دکمه بساز با -ورود-
نمایش پنجره
```

نتیجهٔ بررسی

برنامه صحیح است.

widgets.t — examples2/gui/widgets.t

```
پنجره بساز با -ویجت‌های تیروتیر- و ۶۰۰ و ۶۰۰
برچسب بساز با -چک‌باکس و فهرست-
چک‌باکس بساز با -فعال-
لیست بساز با -قرمز- و -سبز- و -آبی-
منوی انتخاب بساز با -قرمز- و -سبز- و -آبی-
متن چندخطی بساز
نوار لغزش بساز با ۱۰۰
نمایش پنجره
```

نتیجهٔ بررسی

برنامه صحیح است.

۱۱-۴ اجزای دیگر

دستور	جزء
چک‌باکس بساز با -فعال-	کادر تیک‌خور
لیست بساز با -قرمز- و -سبز- و -آبی-	فهرستی از گزینه‌ها
منوی انتخاب بساز با -قرمز- و -سبز-	منوی کشویی
متن چندخطی بساز	کادر نوشتن متن بلند
نوار لغزش بساز با ۱۰۰	لغزنده‌ای از صفر تا ۱۰۰

نکته

در نسخهٔ ۲ رابط گرافیکی تیروتیر در سطح «ساخت و نمایش» است: **سیستم کامل رویداد** (مثلاً کار کردنِ دکمه با کلیک) و اتصال ورودی به متغیر هنوز وجود ندارد. پس «ماشین‌حساب» و «شمارنده»های مخزن ظاهر برنامه را می‌سازند، نه منطقش را. این رابط بر پایهٔ Tkinter است.

۱۱-۵ اصول ساده‌ی طراحی

- **خوانایی:** متن‌ها کوتاه و روشن باشد.
- **فاصله‌گذاری:** اجزا به هم نچسبند و منظم زیر هم بیایند.
- **تناسب اندازه:** اندازهٔ پنجره به اندازهٔ اجزا باشد؛ نه خیلی بزرگ و نه کوچک.
- **از دید کاربر فکر کنید:** از یک دوست بخواهید پنجره را ببیند و بازخورد بدهد.

مثال‌های آمادهٔ این فصل در گیتهاب

پوشهٔ **examples2/gui** را در مخزن رسمی ببینید و فایل‌ها را بخوانید و اجرا کنید:

```
gui/hello_window.t
gui/login_form.t
gui/form.t
gui/widgets.t
gui/calculator.t
gui/unit_converters.t
```

https://github.com/worker2025/tirotir/tree/main/examples2/gui

مطالعهٔ آزاد: پایتون

پایتون هم Tkinter دارد؛ تیروتیر روی همین کتابخانه بنا شده. در پایتون می‌شود به دکمه «واکنش» داد (`command`) که در نسخهٔ ۲ تیروتیر هنوز نیست:

```
import tkinter as tk

def on_click():
    label.config(text="Hello " + entry.get())

root = tk.Tk()
root.title("My app")
root.geometry("400x200")
label = tk.Label(root, text="Your name:")
label.pack(pady=8)
entry = tk.Entry(root)
entry.pack(pady=8)
button = tk.Button(root, text="Say hello", command=on_click)
button.pack(pady=8)
root.mainloop()
```

این برنامه به صفحه‌نمایش یا دستگاه صوتی نیاز دارد؛ روی رایانهٔ خودتان اجرا کنید.

این برنامه روی رایانهٔ شما پنجره باز می‌کند و با کلیک دکمه، سلام می‌دهد.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. کدام یک «جزء رابط کاربری» نیست؟

الف) دکمه　ب) برچسب　ج) کادر ورودی　د) حلقهٔ تکرار

۲. آخرین دستور یک برنامهٔ پنجره‌ای چیست؟

الف) پنجره بساز　ب) نمایش پنجره　ج) دکمه بساز　د) چاپ

۳. برای اجرای یک برنامهٔ GUI در خط فرمان از چه دستوری استفاده می‌کنیم؟

الف) tirotir gui app.t　ب) tirotir graphics app.t　ج) tirotir ast app.t　د) tirotir repl

۴. GUI تیروتیر بر پایهٔ چه کتابخانه‌ای است؟

الف) Tkinter　ب) SVG　ج) WAV　د) PyInstaller

۵. کدام ویژگی هنوز در GUI نسخهٔ ۲ نیست؟

الف) ساخت دکمه　ب) ساخت پنجره　ج) سیستم کامل رویداد (واکنش دکمه به کلیک)　د) کادر ورودی

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. آسان پنجره‌ای با عنوان «برنامهٔ من» بسازید که یک برچسب و یک دکمه داشته باشد (فقط بررسی کنید).
2. متوسط پنجرهٔ «کارت معرفی» با نام، شهر و دکمهٔ ثبت بسازید.
3. متوسط یک «کارت تبریک» برای مناسبت دلخواه بسازید.
4. چالشی پنجره‌ای شامل برچسب، ورودی، چک‌باکس، منوی انتخاب و دکمه بسازید و طرح کاغذی آن را بکشید و مقایسه کنید.

فصل ۱۲

بررسی برنامه و رفع خطا

در پایان این فصل می‌توانید:

- انواع خطا (نوشتاری، معنایی، منطقی) را بشناسید
- پیام‌های خطای فارسی تیروتیر را بخوانید و محل خطا را پیدا کنید
- از `check` و `ast` کمک بگیرید
- با چاپ مرحله‌ای، مقدار متغیرها را ردیابی کنید
- خطا را درست گزارش کنید

۱۲-۱ خطا طبیعی است

همهٔ برنامه‌نویس‌ها خطا می‌کنند؛ حتی حرفه‌ای‌ها! مهم این است که پیام خطا را بخوانیم و رفعش کنیم. پیام‌های تیروتیر فارسی‌اند و اغلب پیشنهادِ اصلاح هم می‌دهند. خطاها سه دسته‌اند:

- **خطای نوشتاری:** برنامه را اشتباه نوشته‌ایم (املا، تورفتگی، علامت ناشناخته).
- **خطای معنایی:** نوشتار درست است ولی معنا ندارد (متغیری که ساخته نشده، تعداد ورودی نادرست).
- **خطای منطقی:** برنامه بدون پیام خطا اجرا می‌شود ولی جواب غلط می‌دهد.

فعالیت بدون رایانه

برنامهٔ معیوب. هر گروه یک «دستور پخت» یا «مسیر رفتن به مدرسه» را با عمد خراب می‌نویسد (یک مرحله جا بیفتد، ترتیب عوض شود). گروه دیگر خطا را پیدا و درست می‌کند.

۱۲-۲ خطای نوشتاری

typo.t

```
چپ -سلام-
```

خروجی

```
خطا: انتظار یک عبارت داشتم، اما «» دریافت شد (خط 1).
```

«چاپ» را «چپ» نوشتیم. رایانه «چپ» را نمی‌شناسد. وقتی پیام زیاد گویا نیست، خط را با دقت دوباره بخوانید. نمونهٔ دیگر، تورفتگی:

indent.t

```
بگذار x برابر ۵
اگر ۳ < x
چاپ -بزرگ-
```

خروجی

```
خطا: خطای نحوی در خط 3: بدنه باید تورفته باشد.
```

equal.t

```
بگذار ۵ x
```

خروجی

```
خطا: انتظار «برابر» در خط 1 داشتم.
```

badchar.t

```
چاپ ۲ ^ ۳
```

خروجی

```
خطا: نویسهٔ ناشناخته «^» در خط 1، ستون 7
```

پیام می‌گوید «نویسهٔ ناشناخته» و حتی شمارهٔ خط و ستون را می‌دهد. در تیروتیر توان با `^` نیست؛ فقط ضرب را داریم.

۱۲-۳ خطای معنایی: کد خطا و پیشنهاد

این پیام‌ها کد دارند (مانند `TIR2001`) و علاوه بر توضیح، «پیشنهاد» هم می‌دهند:

undefined.t

```
چاپ سن
```

خروجی

```
TIR2001: متغیر «سن» پیدا نشد.
پیشنهاد: ابتدا آن را با «بگذار» تعریف کن.
```

args.t

```
تابع f با الف
  بازگردان الف
چاپ f با ۱ و ۲
```

خروجی

```
TIR2007: تعداد آرگومان‌های تابع «f» نادرست است.
پیشنهاد: 1 آرگومان لازم است.
```

const.t

```
ثابت س برابر ۱
س = ۲
```

خروجی

```
TIR2003: ثابت «س» قابل تغییر نیست.
پیشنهاد:
```

کد	معنی
TIR2001	نامی (متغیر یا تابع) پیدا نشد
TIR2002	نام در همین محدوده قبلاً تعریف شده است
TIR2003	ثابت قابل تغییر نیست
TIR2004	«بازگردان» بیرون از تابع
TIR2005 / TIR2006	«بیرون» / «ادامه» بیرون از حلقه
TIR2007	تعداد ورودی‌های تابع نادرست
TIR3001 تا TIR3003	خطاهای ماژول و مسیر (چرخه، پیدا نشدن، مسیر نامعتبر)
TIR4001	رابط گرافیکی در این رایانه در دسترس نیست

توجه

گاهی پیام، علت اصلی را مستقیم نمی‌گوید. مثلاً اگر متنی را ببندید ولی خط‌تیرهٔ پایانی را فراموش کنید، تیروتیر ممکن است «متغیر پیدا نشد» بگوید:

unclosed.t

```
چاپ -سلام
```

خروجی

```
TIR2001: متغیر «سلام» پیدا نشد.
پیشنهاد: ابتدا آن را با «بگذار» تعریف کن.
```

۱۲-۴ خطاهای زمان اجرا

بعضی اشتباه‌ها فقط وقتی برنامه به آن‌ها می‌رسد معلوم می‌شود:

type.t

```
چاپ -سن: - + ۵
```

خروجی

```
خطا: عملیات «+» روی این نوع داده مجاز نیست.
```

zero.t

```
چاپ ۱ / ۰
```

خروجی

```
خطا: عملیات «/» روی این نوع داده مجاز نیست.
```

index.t

```
بگذار ل برابر [۱، ۲]
چاپ ل[۵]
```

خروجی

```
خطا: دسترسی به فهرست یا فرهنگ نامعتبر است.
```

نکته

دستور `tirotir check` این‌گونه خطاهای زمان اجرا را **نمی‌بیند** و برای هر سه برنامهٔ بالا می‌گوید «برنامه صحیح است». `check` فقط نوشتار و نام‌ها را می‌سنجد؛ پس بررسی‌شدن یعنی «احتمالاً درست»، نه «حتماً درست». حتماً برنامه را هم اجرا و آزمایش کنید.

zero.t

```
چاپ ۱ / ۰
```

نتیجهٔ بررسی

```
برنامه صحیح است.
```

۱۲-۵ خطای منطقی

میانگین ۱۵ و ۱۷ و ۲۰ را حساب می‌کنیم. جواب باید ۱۷٫۳۳ باشد:

logic.t

```
بگذار میانگین برابر ۱۵ + ۱۷ + ۲۰ / ۳
چاپ میانگین
```

خروجی

```
38.666666666666664
```

خطایی نداد، ولی جواب غلط است! چون ضرب و تقسیم پیش از جمع انجام می‌شود. با پرانتز درستش می‌کنیم:

logic_fixed.t

```
بگذار میانگین برابر (۱۵ + ۱۷ + ۲۰) / ۳
چاپ میانگین
```

خروجی

```
17.333333333333332
```

۱۲-۶ راهبرد رفع اشکال

1. **بازتولید:** خطا را دوباره ایجاد کنید و ببینید همیشه رخ می‌دهد یا نه.
2. **حدس:** بگویید کدام خط یا متغیر مشکوک است.
3. **آزمون:** با «چاپ» مقدار متغیرها را ردیابی کنید.
4. **اصلاح:** یک تغییر کوچک بدهید و دوباره بسنجید.

trace.t

```
بگذار مجموع برابر ۰
برای عدد در [۴، ۵، ۶]
    بگذار مجموع برابر مجموع + عدد
    چاپ مجموع
```

خروجی

```
4
9
15
```

چاپ مرحله‌ای نشان می‌دهد مجموع در هر دور چطور رشد می‌کند.

۱۲-۷ ابزارها و گزارش خطا

- `tirotir check`: بررسی بدون اجرا.
- `tirotir ast`: نمایش درخت ساختار برنامه (در حد آشنایی).
- `tirotir format`: قالب‌بندی کد؛ در نسخهٔ ۲ هنوز ساده است و تغییر چندانی نمی‌دهد.

اگر فکر می‌کنید خطا از خود تیروتیر است، آن را در بخش Issues مخزن رسمی گزارش کنید. این اطلاعات را بنویسید:

- نسخهٔ زبان و افزونه (`tirotir version`)
- سیستم‌عامل و نسخهٔ پایتون
- فایل `.t` کوچکی که خطا را نشان می‌دهد
- خروجی مورد انتظار و خروجی واقعی
- پیام خطا

ایمنی و سلامت

در رفع اشکال طولانی به استراحت دوره‌ای و ارگونومی توجه کنید و برای بررسی‌ها از چاپ کاغذی استفاده نکنید؛ روی صفحه ببینید.

مطالعهٔ آزاد: پایتون

پایتون هم پیام خطا می‌دهد (به انگلیسی) و آن را «Traceback» می‌گویند که خط خطا را نشان می‌دهد. پایتون برخلاف تیروتیر خطاهای زمان اجرا را با `try / except` می‌تواند بگیرد:

```
try:
    print(1 / 0)
except ZeroDivisionError:
    print("cannot divide by zero")

items = [1, 2]
try:
    print(items[5])
except IndexError:
    print("no such item")

print(15 + 17 + 20 / 3)
print((15 + 17 + 20) / 3)
```

خروجی:

```
cannot divide by zero
no such item
38.666666666666664
17.333333333333332
```

تیروتیر ۲ هنوز مدل عمومی استثنا (`try/except`) ندارد.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. اگر برنامه بدون پیام خطا اجرا شود ولی جواب غلط بدهد، چه نوع خطایی دارد؟

الف) نوشتاری ب) معنایی ج) منطقی د) هیچ‌کدام

۲. پیام `TIR2001` دربارهٔ چیست؟

الف) نام (متغیر یا تابع) پیدا نشد ب) تقسیم بر صفر ج) تورفتگی نادرست د) نبودن Tkinter

۳. آیا `tirotir check` می‌تواند «تقسیم بر صفر» را پیش از اجرا پیدا کند؟

الف) بله همیشه ب) نه؛ این خطا فقط موقع اجرا پیدا می‌شود ج) فقط در حلقه‌ها د) فقط در توابع

۴. کدام گام اولِ راهبردِ رفع اشکال است؟

الف) حذف همهٔ برنامه ب) بازتولید خطا ج) تغییر زبان د) خاموش‌کردن رایانه

۵. در گزارش خطا کدام اطلاعات لازم نیست؟

الف) نسخهٔ تیروتیر ب) خروجی واقعی ج) رمز عبور حساب‌ها د) فایل کوچک بازتولیدکننده

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. **آسان** برنامهٔ زیر سه خطا دارد. آن‌ها را پیدا و درست کنید:

```
بگذار نام برابر -سارا-
بگذار نمره برابر ۱۷
اگر نمره بزرگتر یا مساوی ۱۰
چاپ -قبول-
وگرنه
  چاپ -مردود-
چاپ نام + -: - + نمره
```

2. **متوسط** این برنامه خطای منطقی دارد؛ می‌خواهیم میانگین دو نمرهٔ ۱۲ و ۱۸ را (باید ۱۵) حساب کند. درستش کنید:

```
بگذار میانگین برابر ۱۲ + ۱۸ / ۲
چاپ میانگین
```

3. **متوسط** برنامه‌ای بنویسید که فهرست [۳، ۶، ۹] را با حلقه جمع کند و مقدار مجموع را پس از هر دور چاپ کند (ردیابی).

4. **چالشی** برنامه‌ای با یک خطای عمدی برای هم‌گروهی‌تان بنویسید و سپس برنامهٔ او را رفع اشکال کنید. گزارش خطا را با سه خط (نسخه، خروجی مورد انتظار، خروجی واقعی) بنویسید. (پاسخ نمونه: یک برنامهٔ درست.)

فصل ۱۳

ساخت برنامهٔ اجرایی با Tirotir Builder

در پایان این فصل می‌توانید:

- برنامهٔ اجرایی (EXE) را از اجرای معمولی با مفسر تشخیص دهید
- پیش از ساخت، برنامه را با `check` و اجرا آزمایش کنید
- با `tirotir builder` فایل اجرایی بسازید
- سه فایل `tirotir.exe`، `program.exe` و Setup تاریخی را از هم تفکیک کنید
- اصول توزیع امن برنامه را رعایت کنید

۱۳-۱ EXE چیست؟

تا اینجا برنامه‌ها را با دستور `tirotir run` اجرا می‌کردیم؛ یعنی برای اجرا، به تیروتیر نصب‌شده نیاز داشتیم. اگر بخواهید برنامه‌تان را به کسی بدهید که تیروتیر ندارد، می‌توانید از آن یک «**برنامهٔ اجرایی**» (فایل `.exe` در ویندوز) بسازید که با دو بار کلیک اجرا می‌شود.

توجه

ساخت فایل اجرایی بومی ویندوز باید **روی خودِ ویندوز** انجام شود. روی لینوکس یا مک، خروجی همان سیستم ساخته می‌شود. اعتبارسنجی کامل بستهٔ ویندوز و فایل اجرایی در نسخهٔ ۲ هنوز کار آینده است؛ پس مربی یا خودتان، فایل ساخته‌شده را روی یک رایانهٔ ویندوز واقعی آزمایش کنید.

۱۳-۲ آماده‌سازی برنامه

پیش از ساخت، برنامه باید کاملاً درست باشد. یک برنامهٔ کوچک می‌نویسیم و آن را هم اجرا و هم بررسی می‌کنیم:

program.t

```
چاپ -نام شما چیست؟-
بگذار نام برابر بخوان
بگذار سلام برابر -سلام -
بگذار پایان برابر -! برنامهٔ من آماده است.-
چاپ سلام + نام + پایان
```

خروجی

```
ورودی تایپ‌شده: سارا
نام شما چیست؟
سلام سارا! برنامهٔ من آماده است.
```

program.t

```
چاپ -نام شما چیست؟-
بگذار نام برابر بخوان
بگذار سلام برابر -سلام -
بگذار پایان برابر -! برنامهٔ من آماده است.-
چاپ سلام + نام + پایان
```

نتیجهٔ بررسی

```
برنامه صحیح است.
```

اگر برنامه‌ای «بخوان» داشته باشد، فایل ساخته‌شده هم از کاربر ورودی می‌گیرد؛ پس هنگام آزمایش، ورودی را هم امتحان کنید.

۱۳-۳ ساخت با Tirotir Builder

ابزار ساخت را با این دستور باز کنید (روی ویندوز):

خط فرمان

```
python -m pip install pyinstaller
tirotir builder
```

Tirotir Builder از ابزار **PyInstaller** استفاده می‌کند؛ پس باید یک‌بار نصبش کنید. پنجرهٔ «Tirotir EXE Builder» باز می‌شود. این بخش‌ها را می‌بینید:

- **فایل اصلی (.t):** با دکمهٔ «انتخاب» برنامهٔ خود را نشان دهید.
- **نام برنامه:** نام فایل خروجی (مثلاً `program` تا `program.exe` ساخته شود).
- **حالت خروجی:** `One File` (یک فایل) یا `Folder` (یک پوشه).
- **نمایش کنسول:** برای برنامه‌های متنی و دارای «بخوان» روشن بماند.
- **آیکون و مسیر خروجی:** اختیاری؛ پیش‌فرض مسیر خروجی پوشهٔ `dist` است.
- **Build EXE:** ساخت را شروع می‌کند. در کادر پایین پیشرفت کار نوشته می‌شود و در پایان، فایل ساخته‌شده یک‌بار اجرا می‌شود.
- **باز کردن پوشهٔ خروجی:** فایل ساخته‌شده را نشانتان می‌دهد.

نکته

خود Tirotir Builder هم یک برنامهٔ دارای پنجره (Tkinter) است؛ پس مثل فصل ۱۱ به صفحه‌نمایش واقعی نیاز دارد و در محیط بدون صفحه‌نمایش باز نمی‌شود.

۱۳-۴ سه فایل مهم

فایل	چیست؟
tirotir.exe	اجرایی اصلی خودِ زبان و خط فرمان (CLI)
program.exe	خروجی برنامه‌ای که شما با تیروتیر نوشته‌اید؛ نامش همانی است که در «نام برنامه» داده‌اید
Tirotir-1.1.0-Setup.exe	نصب‌کنندهٔ تاریخیِ نسخهٔ ۱؛ نسخهٔ ۲ با بستهٔ پایتون (pip) نصب می‌شود

۱۳-۵ تحویل و توزیع امن

- فایل را با نامی روشن بنامید و در یک پوشهٔ منظم (مثلاً با یک فایل توضیح کوتاه) تحویل دهید.
- منبع برنامه را معرفی کنید (نویسنده، تاریخ) و مجوز کدهایی را که از دیگران برداشته‌اید رعایت کنید.
- **فقط فایل‌های ساخته‌شدهٔ خودتان و منابع مطمئن** را اجرا کنید؛ فایل ناشناخته را اجرا نکنید و از ضدویروس استفاده کنید.
- پیش از تحویل، برنامه را روی یک رایانهٔ دیگر آزمایش کنید.

محیط‌زیست و فضای ذخیره‌سازی

فایل‌های آزمایشیِ اضافه را پس از پایان کار حذف کنید. بیشتر فایل‌های پوشهٔ ساخت موقت هستند.

مطالعهٔ آزاد: پایتون

در پایتون هم ابزارهایی برای ساخت برنامهٔ اجرایی هست؛ معروف‌ترینشان همان PyInstaller است که Tirotir Builder از آن استفاده می‌کند:

```
# در ترمینال:
#   python -m pip install pyinstaller
#   pyinstaller --onefile hello.py
# ساخته می‌شود dist فایل اجرایی در پوشهٔ
print("Hello from an executable")
```

این برنامه به صفحه‌نمایش یا دستگاه صوتی نیاز دارد؛ روی رایانهٔ خودتان اجرا کنید.

ساختن برنامهٔ اجرایی روی ویندوز، فایل `.exe` می‌دهد و روی لینوکس/مک فایل اجرایی همان سیستم را.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. مزیت برنامهٔ اجرایی (EXE) چیست؟

الف) به رایانه برق نمی‌رساند ب) بدون نصب تیروتیر هم قابل اجراست ج) سرعت اینترنت را بالا می‌برد د) برنامه را رنگی می‌کند

۲. ساخت فایل اجرایی بومی ویندوز باید روی چه سیستمی انجام شود؟

الف) هر سیستمی ب) ویندوز ج) فقط لینوکس د) فقط مک

۳. قبل از ساخت EXE چه کاری لازم است؟

الف) برنامه را بدون بررسی بسازیم ب) برنامه را با check و اجرا آزمایش کنیم ج) همهٔ فایل‌ها را حذف کنیم د) نام برنامه را فاصله‌دار کنیم

۴. `Tirotir-1.1.0-Setup.exe` چیست؟

الف) نصب‌کنندهٔ تاریخی نسخهٔ ۱ ب) برنامهٔ شما ج) اجرایی اصلی نسخهٔ ۲ د) افزونهٔ VS Code

۵. کدام کار در توزیع امن برنامه درست است؟

الف) اجرای هر فایل ناشناخته ب) معرفی منبع و رعایت مجوز ج) حذف نام نویسنده د) غیرفعال‌کردن ضدویروس

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. آسان برنامهٔ «سلام» فصل را بنویسید و با `tirotir check` بررسی کنید.
2. آسان همان برنامه را با یک نام دلخواه اجرا و آزمایش کنید و خروجی را یادداشت کنید.
3. متوسط با `tirotir builder` فایل اجرایی بسازید و نقش هر یک از سه فایل مهم را بنویسید. (تمرین عملی روی ویندوز؛ پاسخ در متن فصل است.)
4. چالشی برنامه‌ای کامل (مثلاً ماشین‌حساب جمع دو عدد) بنویسید، آزمایش کنید، EXE بسازید و روی رایانهٔ دوستتان بیازمایید.

فصل ۱۴

پروژهٔ چندفایلی، قابلیت‌ها و ارائهٔ پروژهٔ پایانی

در پایان این فصل می‌توانید:

- ساختار پروژه را (فایل پروژه، فایل اصلی و پوشهٔ lib) بشناسید
- با «وارد کن» ماژول را به پروژه بیاورید
- قابلیت‌های graphics، audio و gui را آگاهانه روشن کنید
- پروژهٔ پایانی را برنامه‌ریزی، پیاده‌سازی و آزمایش کنید
- پروژه‌تان را در حداکثر پنج دقیقه ارائه دهید

۱۴-۱ پروژهٔ چندفایلی

برنامهٔ بزرگ در یک فایل خوانا نیست. در یک «پروژه»، کار بین چند فایل تقسیم می‌شود. هر پروژه یک پوشه است با این ساختار:

```
my-project/
├── tirotir.toml     ← فایل پروژه (نام، فایل اصلی، قابلیت‌ها)
├── main.t           ← فایل اصلی
└── lib/
    └── tools.t      ← یک ماژول
```

فایل پروژه `tirotir.toml` این‌گونه نوشته می‌شود:

tirotir.toml

```
[project]
name = "my-project"
entry = "main.t"
language = "2.0"

[capabilities]
graphics = false
audio = false
gui = false
project-files = false
network = false
```

- `entry`: فایلی که اجرا از آن شروع می‌شود.
- `[capabilities]`: دسترسی‌های ویژه که به‌طور پیش‌فرض همه **خاموش**اند (بخش ۱۴-۳).

۱۴-۲ ماژول و «وارد کن»

هر فایل دیگرِ پروژه یک «ماژول» است. با `وارد کن -مسیر` آن را به برنامه می‌آوریم (مسیر نسبت به ریشهٔ پروژه است و پسوند `.t` را نمی‌نویسیم). این پروژهٔ دوفایلی را ببینید:

tirotir.toml

```
[project]
name = "karname"
entry = "main.t"
language = "2.0"

[capabilities]
graphics = false
audio = false
gui = false
project-files = false
network = false
```

lib/title.t

```
چاپ -=== کارنامهٔ کلاس ===-
```

main.t

```
وارد کن -lib/title-
بگذار نمره‌ها برابر [۱۸، ۱۶، ۲۰]
بگذار مجموع برابر ۰
برای نمره در نمره‌ها
    بگذار مجموع برابر مجموع + نمره
چاپ مجموع / ۳
```

خروجی `tirotir run` در ریشهٔ پروژه

```
=== کارنامهٔ کلاس ===

18.0
```

«وارد کن» ماژول را یک‌بار اجرا می‌کند (آن چاپِ عنوان). برنامه را از **ریشهٔ پروژه** فقط با `tirotir run` (بدون نام فایل) اجرا می‌کنیم.

توجه

محدودیت شناخته‌شدهٔ نسخهٔ ۲ (KI-6): «وارد کن» ماژول را پیدا و اجرا می‌کند و امنیت مسیر را رعایت می‌کند (مسیر بیرون از پوشهٔ پروژه یا ماژول ناموجود خطا می‌دهد)؛ اما **تابع یا متغیری که در ماژول ساخته‌اید، فعلاً در فایل اصلی قابل استفاده نیست.** پس فعلاً از ماژول برای آماده‌سازی، عنوان یا چاپ‌های مشترک استفاده کنید. اشتراک تابع بین فایل‌ها هدف نسخه‌های بعدی است. (در پایتون این کار ساده و کامل است؛ به بخش مطالعهٔ آزاد نگاه کنید.)

اگر مسیر ماژول اشتباه باشد، خطای `TIR3002` می‌دهد:

main.t: وارد کن -lib/نیست-

پیام خطا

TIR3002: ماژول پیدا نشد: .../lib/نیست.tirotir

۱۴-۳ قابلیت‌ها (Capabilities)

برنامه‌هایی که گرافیک، صدا، پنجره، فایل یا شبکه می‌خواهند «قابلیت» لازم دارند: `graphics`، `audio`، `gui`، `project-files` و `network`. به‌طور پیش‌فرض همه خاموش‌اند؛ پس اگر قابلیتی را روشن نکنید، برنامه‌ای که آن را لازم دارد می‌ایستد:

needs_graphics.t

پنجره بازکن با ۸۰۰ و ۶۰۰

مربع با ۱۰۰

خروجی

خطا: قابلیت «graphics» فعال نیست.

دو راه برای روشن‌کردن هدفمند یک قابلیت:

- در خط فرمان: `tirotir run app.t --capability graphics`
- در فایل پروژه: `graphics = true` در بخش `[capabilities]`

نمونه: پروژهٔ نقاشی با graphics = true

خروجی tirotir run

نقاشی کشیده شد

و فایل تصویر tirotir-scene.svg کنار پروژه ساخته می‌شود.

ایمنی

قابلیت‌های `network` و `project-files` را بدون نیاز و بدون اجازهٔ مربی روشن نکنید. این سازوکار «کنترل دسترسی» است و جداسازی امنیتی کامل در سطح سیستم‌عامل نیست؛ پس برنامهٔ ناشناخته را اجرا نکنید.

۱۴-۴ پروژهٔ پایانی

حالا همهٔ آموخته‌هایتان را در یک پروژه به کار می‌برید. این مراحل را بروید:

1. **ایده:** چه برنامه‌ای می‌خواهید؟ (جمله‌ای بنویسید: «برنامهٔ من ... می‌کند».)
2. **طرح:** ورودی‌ها، کارها و خروجی‌ها را روی کاغذ بنویسید. چه متغیرها، شرط‌ها، حلقه‌ها و توابعی لازم است؟
3. **ساخت:** گام‌به‌گام بنویسید و بعد از هر گام `check` و اجرا کنید.
4. **آزمایش:** با ورودی‌های مختلف (و ورودی عجیب!) امتحان کنید و خطاها را رفع کنید.
5. **ارائه:** حداکثر ۵ دقیقه؛ ایده، نمایش برنامه، سختی‌ها و یادگیری‌تان را بگویید و به پرسش‌ها جواب دهید.

نمونه ایده‌ها

- بازی «حدس عدد» یا «سنگ‌کاغذقیچی» (متن + شرط + حلقه)

- دفترچهٔ تلفن یا فهرست خرید (فهرست و فرهنگ)
- کارنامهٔ نمره با میانگین و پیام تشویقی (تابع و حلقه)
- یک نقاشی خلاقانه، مثل خانه یا گل (گرافیک)
- یک ملودی کوتاه (صدا)
- یک کارت تبریک پنجره‌ای (GUI)

برگهٔ طرح پروژه

عنوان پروژه	
برنامهٔ من ... می‌کند	
ورودی‌ها	
خروجی‌ها	
مفاهیمی که استفاده می‌کنم	
تقسیم کار (اگر گروهی است)	
منابع و کدهایی که از دیگران گرفتم	

نکته

هنگام ارائه، ذکر منبع نمونه‌کدهای دیگران بخشی از **اخلاق حرفه‌ای** است؛ همین‌طور احترام به هم‌گروهی‌ها، تقسیم عادلانهٔ کار و رعایت نوبت.

یک پروژهٔ نمونه: بازی حدس عدد

guess.t

```
بگذار راز برابر ۷
بگذار فرصت برابر ۳
بگذار برد برابر نادرست
چاپ -عددی بین ۱ تا ۱۰ حدس بزن-
تاوقتی فرصت > ۰ و نه برد
    بگذار متن برابر بخوان
    بگذار حدس برابر به_عدد با متن
    اگر حدس == راز
        چاپ -آفرین! درست حدس زدی-
        بگذار برد برابر درست
    وگرنه اگر حدس > راز
        چاپ -کمتر-
    وگرنه
        چاپ -بیشتر-
    بگذار فرصت برابر فرصت - ۱
اگر نه برد
    چاپ -فرصتت تمام شد-
```

خروجی

ورودی تایپ‌شده: **۵ ، ۹ ، ۷**

```
عددی بین ۱ تا ۱۰ حدس بزن
بیشتر
کمتر
آفرین! درست حدس زدی
```

مطالعهٔ آزاد: پایتون

در پایتون هر فایل `.py` یک ماژول است و با `import` می‌توان تابع‌ها را بین فایل‌ها به اشتراک گذاشت. این کار در تیروتیر ۲ هنوز کامل نیست؛ پس پایتون راه خوبی برای تمرین «پروژهٔ چندفایلی» است. دو فایل `tools.py` و `main.py` بسازید:

```
# فایل tools.py
def double(number):
    return number * 2

# فایل main.py
# from tools import double
# print(double(21))

# شبیه‌سازی در یک فایل:
print(21 * 2)
```

خروجی:

```
42
```

در پروژهٔ واقعی خط‌های commentشده را در فایل‌های جدا بگذارید: `from tools import double`.

آزمونک فصل (۵ پرسش چهارگزینه‌ای؛ پاسخ در انتهای کتاب)

۱. فایل تنظیمات یک پروژهٔ تیروتیر چه نام دارد؟

الف) main.t　ب) tirotir.toml　ج) lib.exe　د) project.svg

۲. فرمان اجرای پروژه از ریشهٔ پوشه چیست؟

الف) tirotir run　ب) tirotir builder　ج) tirotir repl　د) tirotir ast

۳. وضعیت پیش‌فرض قابلیت‌هایی مثل graphics و network چیست؟

الف) روشن　ب) خاموش　ج) فقط روی ویندوز روشن　د) پاک می‌شود

۴. پروژهٔ پایانی را در چند دقیقه ارائه می‌دهیم؟

الف) حداکثر ۵ دقیقه　ب) ۳۰ دقیقه　ج) ۱ ثانیه　د) محدودیت ندارد

۵. در ارائهٔ پروژه، استفاده از کدِ دیگران چطور باید باشد؟

الف) پنهان شود　ب) منبعش ذکر شود　ج) به‌نام خودمان شود　د) مهم نیست

تمرین خانه

هر تمرین را خودتان روی رایانه بنویسید و اجرا کنید؛ سپس با پاسخ‌نامهٔ انتهای کتاب مقایسه کنید. پاسخ درست یکتا نیست: اگر خروجی برنامهٔ شما همان است، برنامه‌تان درست است.

1. آسان طرح پروژهٔ پایانی‌تان را با برگهٔ طرح پروژه پر کنید. (تمرین نوشتاری؛ نمونهٔ ساده در پاسخ آمده.)
2. متوسط یک پروژهٔ دوفایلی بسازید که ماژولی عنوان را چاپ کند و فایل اصلی کاری انجام دهد؛ با `tirotir run` اجرایش کنید. (مانند پروژهٔ «کارنامه» همین فصل.)
3. متوسط بازی «حدس عدد» را طوری تغییر دهید که فرصت ۵ بار باشد.
4. چالشی پروژهٔ پایانی‌تان را بسازید، آزمایش کنید و در ۵ دقیقه ارائه دهید. یک کارت خودارزیابی هم بنویسید: چه چیزی خوب شد؟ چه چیزی را بهتر می‌کنید؟

پیوست آ

دستورهای خط فرمان تیروتیر

فرمان	کاربرد
`tirotir version`	نمایش نسخهٔ نصب‌شده
`tirotir run hello.t`	اجرای یک برنامه
`tirotir check hello.t`	بررسی برنامه بدون اجرا (خطاهای زمان اجرا را نمی‌بیند)
`tirotir ast hello.t`	نمایش درخت ساختار برنامه (آشنایی)
`tirotir format hello.t --write`	قالب‌بندی کد؛ در نسخهٔ ۲ هنوز ساده است
`tirotir graphics shape.t --scene scene.svg`	اجرای برنامهٔ گرافیکی و ذخیرهٔ تصویر SVG
`tirotir audio scale.t --play-audio`	ساخت فایل صوتی و پخش آن
`tirotir run app.t --capability graphics`	اجرا با روشن‌کردن صریح یک قابلیت
`tirotir repl`	محیط تعاملی (خروج با نوشتن «خروج»)
`tirotir gui app.t`	اجرای برنامهٔ دارای پنجره
`tirotir run`	اجرای پروژه از ریشهٔ پوشهٔ پروژه
`tirotir builder`	ابزار ساخت برنامهٔ اجرایی (ویندوز)

اسنیپت‌های افزونهٔ VS Code

سلام، متغیر، چاپ، اگر، تکرار، تاوقتی، برای، تابع، فهرست، فرهنگ، ورودی، پنجره، نت، مربع. (پیشوند را بنویسید، `Ctrl+Space` و سپس `Tab` بزنید.)

فرمان‌های راست‌کلیک افزونه

اجرای فایل · بررسی فایل · نمایش AST · اجرای زنده در پنل فارسی راست‌به‌چپ · نمایش Graphics زنده · پخش Audio · اجرای GUI.

پیوست ب

برگهٔ مقایسه: تیروتیر و پایتون

تیروتیر	پایتون
چاپ -سلام-	`print("Hello")`
# توضیح	`# comment`
بگذار سن برابر ۲۰	`age = 20`
بخوان	`input()`
به_عدد با متن	`float(text)`
به_متن با عدد	`str(number)`
درست / نادرست / تهی	`True / False / None`
اگر ... وگرنه اگر ... وگرنه	`if ... : / elif ... : / else:`
بزرگتر یا مساوی ، کوچکتر از ، == ، !=	`>= , < , == , !=`
و ، یا ، نه	`and , or , not`
تکرار ۵ بار	`for _ in range(5):`
تاوقتی شرط	`while condition:`
برای x در فهرست	`for x in items:`
بیرون ، ادامه	`break , continue`
تابع جمع با الف و ب	`def add(a, b):`
بازگردان الف + ب	`return a + b`
جمع با ۷ و ۵	`add(7, 5)`
[۱، ۲، ۳]	`[1, 2, 3]`
افزودن با فهرست و ۴	`items.append(4)`
طول با فهرست	`len(items)`
حذف با فهرست و ۰	`items.pop(0)`
فهرست[۰]	`items[0]`
بگذار شخص برابر فرهنگ / نام برابر -سارا-	`person = {"name": "Sara"}`
شخص[نام]	`person["name"]`
وارد کن -lib/tools-	`import tools`
پنجره بازکن / حرکت با ۱۰۰ / چرخش با ۹۰	`import turtle / forward(100) / left(90)`
پنجره بساز / برچسب بساز / دکمه بساز	`tkinter: Tk() / Label / Button`

تفاوت‌های مهم

۱) در پایتون پس از شرط، حلقه و تابع **دونقطه** می‌آید؛ در تیروتیر نه. ۲) متن در پایتون بین گیومه و در تیروتیر بین خط‌تیره است. ۳) در پایتون تابع می‌تواند بدون ورودی باشد، در تیروتیر ۲ باید دست‌کم یک ورودی داشته باشد. ۴) پایتون `try/except` و کلاس و ماژول کامل دارد؛ تیروتیر ۲ هنوز ندارد.

پیوست پ

واژه‌نامه

واژه	معنی
الگوریتم	روش گام‌به‌گام برای حل یک مسئله
برنامه	فهرست مرتب دستورها برای رایانه
دستور	یک گام از برنامه (مثلاً چاپ)
مفسر	برنامه‌ای که فایل شما را خط‌به‌خط می‌خواند و اجرا می‌کند
متغیر	جعبه‌ای نام‌دار برای نگه‌داشتن یک مقدار
نوع داده	عدد، متن، درست/نادرست و …
ورودی / خروجی	گرفتن چیزی از کاربر / نمایش چیزی به او
عملگر	علامت محاسبه یا مقایسه (+، *، ==، …)
شرط	تصمیم‌گیری در برنامه با اگر و وگرنه
تورفتگی	فاصلهٔ ابتدای خط؛ نشان می‌دهد کدام خط‌ها داخل یک بلوک‌اند
حلقه	تکرار یک کار
تابع	دستور ساختهٔ برنامه‌نویس؛ ورودی می‌گیرد و می‌تواند نتیجه بدهد
پارامتر / آرگومان	ورودی تعریف‌شدهٔ تابع / مقداری که هنگام فراخوانی می‌دهیم
فهرست	مجموعهٔ مرتب‌شدهٔ مقدارها با شمارهٔ جایگاه از صفر
فرهنگ	مجموعهٔ جفت‌های کلید و مقدار
ماژول	فایلی که به برنامهٔ دیگر وارد می‌شود
قابلیت (Capability)	دسترسی ویژه مثل gui، audio، graphics که باید روشن شود
خطا (Bug)	اشتباه در برنامه که باعث توقف یا جواب غلط می‌شود
رفع اشکال	پیدا کردن و درست کردن خطا
AST	درخت ساختار برنامه
SVG	قالب فایل تصویر برداری که تیروتیر خروجی گرافیکی را در آن می‌نویسد
WAV	قالب فایل صوتی که تیروتیر می‌سازد
GUI	رابط گرافیکی؛ برنامه‌ای با پنجره، دکمه و …
EXE	برنامهٔ اجرایی ویندوز
REPL	محیط تعاملی که دستورها را یک‌به‌یک اجرا می‌کند
اسنیپت	تکه‌کد آمادهٔ ویرایشگر

پیوست ت

محدودیت‌های نسخهٔ ۲٫۰٫۲ در یک نگاه

تیروتیر ۲ در محدودهٔ مستندشده‌اش پایدار است؛ ولی همهٔ ویژگی‌های یک زبان کامل را هنوز ندارد. این فهرست به شما کمک می‌کند خطای خودتان را از محدودیت زبان جدا کنید:

موضوع	وضعیت در نسخهٔ ۲٫۰٫۲
متن ثابت	فقط اول یک عبارت (یا بعد از «برابر»، «با»، «بازگردان»، «اگر» و مانند آن) پذیرفته می‌شود؛ بعد از `+` و `==` و `=` نه. متن ثابت را در متغیر بگذارید. بلافاصله پس از خط‌تیرهٔ آغازین نباید فاصله باشد.
حلقهٔ تکرار با متغیر	`تکرار ۵ بار` درست است؛ با متغیر بنویسید `تکرار ن` (بدون «بار»).
تابع بدون ورودی	پشتیبانی نمی‌شود؛ تابع باید دست‌کم یک ورودی داشته باشد.
فراخوانی با عبارت	`به_عدد با س > ۵` بدون پرانتز درست نیست؛ اول نتیجه را در متغیر بریزید یا پرانتز بگذارید.
حلقه روی فرهنگ	حلقهٔ «برای» فقط روی فهرست و متن می‌چرخد.
ماژول	«وارد کن» ماژول را اجرا می‌کند ولی تابع و متغیرهای آن را به فایل اصلی نمی‌آورد (KI-6).
خروجی عددها	تقسیم و `به_عدد` نتیجهٔ اعشاری می‌دهند (4.0)؛ درست/نادرست به‌شکل True/False چاپ می‌شود.
رویدادها و انیمیشن	نحو عمومی رویداد، کلیک دکمه و انیمیشن کامل هنوز نیست. شکل‌های گرافیکی فقط خط‌کشی‌اند.
GUI و صدا	به Tkinter و نمایشگر واقعی، و پخش صدا به دستگاه صوتی نیاز دارد.
ابزارها	LSP کامل، اشکال‌زدای حرفه‌ای و تکمیل خودکار پیشرفته هنوز نیست؛ `tirotir format` فعلاً کار چندانی نمی‌کند.
زبان	کلاس و شیء، مدل عمومی استثنا (try/except)، Async، سیستم بستهٔ کامل ندارد.
امنیت	قابلیت‌ها کنترل دسترسی‌اند، نه جداسازی کامل سیستم‌عامل.
ویندوز EXE	ساخت و اعتبارسنجی کامل آن روی رایانهٔ ویندوز واقعی انجام شود.

پیوست ث

منابع و پیوندهای مهم

سایت رسمی تیروتیر

https://tirotir.ir/

مخزن و مثال‌ها (GitHub)

https://github.com/worker2025/tirotir

پوشهٔ مثال‌های آموزشی (examples2)

https://github.com/worker2025/tirotir/tree/main/examples2

آزمایشگاه یادگیری تعاملی

https://worker2025.github.io/tirotir-learning-lab/

استناد (DOI)

https://doi.org/10.5281/zenodo.22874866

- گزارش وضعیت پایدار: docs/tirotir-2-stable-release-fa.md در مخزن.
- محدودیت‌ها و مسائل شناخته‌شده: known-issues-fa.md در مخزن.
- گزارش خطا: بخش Issues مخزن؛ نسخه‌ها، سیستم‌عامل، فایل `.t` کوچک، خروجی مورد انتظار و واقعی را بنویسید.
- مجوز کد تیروتیر: MIT. نام و نشان تیروتیر متعلق به پروژهٔ رسمی است.

نکتهٔ QR

نشانی‌های روی کتاب را با دوربین گوشی بخوانید. اگر پیوندی باز نشد، نشانی زیر آن را در مرورگر بنویسید.

پاسخ‌نامه

پاسخ آزمونک‌ها و تمرین‌های خانه

پیش از دیدن پاسخ

اول خودتان تلاش کنید. پاسخ تمرین‌ها یکتا نیست؛ اگر خروجی برنامهٔ شما با خروجی نوشته‌شده یکی است، برنامه‌تان هم درست است. همهٔ کدهای زیر با تیروتیر ۲٫۰٫۲ اجرا شده‌اند.

فصل ۱: آشنایی با برنامه‌نویسی و محیط تیروتیر

پاسخ آزمونک

1. **۱ — الف.** فهرست مرتب دستورها «برنامه» است.
2. **۲ — ب.** برنامه‌های تیروتیر با پسوند `.t` ذخیره می‌شوند.
3. **۳ — ج.** `check` فقط بررسی می‌کند.
4. **۴ — ج.** مثال: `چاپ -سلام` .
5. **۵ — ب.** کلمهٔ «خروج» را بنویسید.

تمرین ۱

```
چاپ -نام من سارا است-
چاپ -من در شیراز زندگی می‌کنم-
چاپ -به نقاشی علاقه دارم-
```

خروجی

```
نام من سارا است
من در شیراز زندگی می‌کنم
به نقاشی علاقه دارم
```

هر متن اگر معنادار و بین دو خط‌تیره باشد درست است.

تمرین ۲

```
چاپ -تیروتیر-
چاپ -تیروتیر-
چاپ -تیروتیر-
چاپ -تیروتیر-
```

خروجی

```
تیروتیر
تیروتیر
تیروتیر
تیروتیر
```

در فصل ۶ یاد می‌گیرید این کار با «تکرار» خیلی کوتاه‌تر می‌شود.

تمرین ۳

```
# این برنامه خودم را معرفی می‌کند
چاپ -نام من سارا است-
چاپ -من در شیراز زندگی می‌کنم-
```

خروجی

```
نام من سارا است
من در شیراز زندگی می‌کنم
```

خط‌هایی که با # شروع شوند «توضیح» هستند و رایانه آن‌ها را نادیده می‌گیرد؛ در خروجی نمی‌آیند.

تمرین ۴

```
چاپ -فروش کتاب قصه-
چاپ -کتاب‌ها سالم و تمیز هستند-
چاپ -تماس: ۰۹۱۲۰۰۰۰۰۰۰-
```

خروجی

```
فروش کتاب قصه
کتاب‌ها سالم و تمیز هستند
تماس: ۰۹۱۲۰۰۰۰۰۰۰
```

شمارهٔ تماس را فرضی بنویسید؛ شمارهٔ واقعی خودتان یا دیگران را در برنامه نگذارید.

فصل ۲: کار با VS Code، افزونهٔ تیروتیر و اسنیپت‌ها

پاسخ آزمونک

1. **۱ — الف.** نام افزونه `tirotirLang` است.
2. **۲ — الف.** پیشوند، سپس `Ctrl+Space` و بعد `Tab`.
3. **۳ — ب.** پیشوند «اگر» است.
4. **۴ — ب.** AST درخت ساختار برنامه است.
5. **۵ — ج.** LSP کامل و اشکال‌زدای حرفه‌ای هنوز عرضه نشده.

تمرین ۱

```
چاپ -سلام دنیا-
```

خروجی

```
سلام دنیا
```

اگر پس از Tab، کد آمادهٔ مشابهی درج شد، اسنیپت درست کار کرده است.

تمرین ۲

```
بگذار نام برابر -سارا-
چاپ نام
```

خروجی

```
سارا
```

اسنیپت «متغیر» ساختار `بگذار ... برابر ...` را درج می‌کند.

تمرین ۳

```
بگذار نام برابر -سارا-
چاپ نام
```

نتیجهٔ بررسی

برنامه صحیح است.

باید پیام «برنامه صحیح است» دیده شود.

تمرین ۴

```
چاپ -سلام دنیا-
بگذار سن برابر ۱۰
چاپ سن
اگر سن بزرگتر یا مساوی ۱۸
    چاپ -بزرگسال-
وگرنه
    چاپ -کودک-
تکرار ۳ بار
    چاپ -تیروتیر-
```

خروجی

```
سلام دنیا
10
کودک
تیروتیر
تیروتیر
تیروتیر
```

نمونه‌ای از ترکیب پنج اسنیپت؛ برنامهٔ شما می‌تواند متفاوت باشد.

فصل ۳: متغیر، ورودی و خروجی

پاسخ آزمونک

1. **۱ — ب.** متغیر `سن` ساخته و مقدار ۲۰ را در آن گذاشته می‌شود.
2. **۲ — ب.** متن بین دو خطتیره می‌آید.
3. **۳ — ب.** ورودی همیشه متن است.
4. **۴ — ج.** ۴ + ۱ = ۵.
5. **۵ — ب.** `به_متن با ...` عدد را به متن تبدیل می‌کند.

تمرین ۱

```
بگذار نام برابر -پویا-
بگذار سن برابر ۹
بگذار دانش‌آموز برابر درست
چاپ نام
چاپ سن
چاپ دانش‌آموز
```

خروجی

```
پویا
9
True
```

درست/نادرست به‌شکل True/False چاپ می‌شود.

تمرین ۲

```
چاپ -نامت چیست؟-
بگذار نام برابر بخوان
بگذار پایان برابر -، خوش آمدی-
چاپ -سلام - + نام + پایان
```

خروجی

ورودی: **مینا**

```
نامت چیست؟
سلام مینا، خوش آمدی
```

نام را با + به متن‌های ثابت چسباندیم؛ متن دوم را اول در یک متغیر گذاشتیم.

تمرین ۳

```
چاپ -نام؟-
بگذار نام برابر بخوان
چاپ -سن؟-
بگذار سن برابر بخوان
چاپ -شهر؟-
بگذار شهر برابر بخوان
چاپ -نام: - + نام
چاپ -سن: - + سن
چاپ -شهر: - + شهر
```

خروجی

ورودی: **علی ، ۱۰ ، تبریز**

```
نام؟
سن؟
شهر؟
نام: علی
سن: ۱۰
شهر: تبریز
```

چون «بخوان» متن می‌دهد، سن را هم می‌توان مستقیم به متن چسباند.

تمرین ۴

```
بگذار امتیاز برابر ۱۰
بگذار امتیاز برابر امتیاز + ۵
چاپ امتیاز
بگذار امتیاز برابر امتیاز + ۵
چاپ امتیاز
```

خروجی

```
15
20
```

خروجی: ۱۵ و بعد ۲۰.

فصل ۴: محاسبات و عبارت‌های ریاضی

پاسخ آزمونک

1. **۱ — ب.** ضرب اول: ۳ × ۴ = ۱۲، بعد ۲ + ۱۲ = ۱۴.
2. **۲ — الف.** پرانتز اول: ۵ × ۴ = ۲۰.
3. **۳ — ب.** تقسیم همیشه اعشاری است: `4.0`.
4. **۴ — ج.** `%`؛ مثلاً ۱۰ % ۴ برابر ۲ است.
5. **۵ — ج.** `... به_عدد با`.

تمرین ۱

```
بگذار الف برابر ۲۴
بگذار ب برابر ۶
چاپ الف + ب
چاپ الف - ب
چاپ الف * ب
چاپ الف / ب
```

خروجی

```
30
18
144
4.0
```

خروجی‌ها: ۳۰، ۱۸، ۱۴۴ و 4.0.

تمرین ۲

```
بگذار طول برابر ۱۲
بگذار عرض برابر ۷
چاپ "مساحت: " + به_متن با طول * عرض
```

خروجی

```
مساحت: 84
```

اگر به‌جای عبارت داخل «به_متن» ایراد دیدید، اول نتیجه را در متغیر بریزید و بعد آن را به متن تبدیل کنید.

تمرین ۳

```
بگذار میانگین برابر (۱۵ + ۱۷ + ۲۰) / ۳
چاپ میانگین
```

خروجی

```
17.333333333333332
```

حاصل ۵۲ تقسیم بر ۳ است: ۱۷٫۳۳۳... (در خروجی چند رقم اعشار می‌بینید).

تمرین ۴

```
چاپ -طول؟-
بگذار الف برابر بخوان
چاپ -عرض؟-
بگذار ب برابر بخوان
بگذار طول برابر به_عدد با الف
بگذار عرض برابر به_عدد با ب
چاپ ۲ * (طول + عرض)
```

خروجی

ورودی: ۸ ، ۵

```
طول؟
عرض؟
26.0
```

۲ × (۸ + ۵) = ۲۶؛ خروجی 26.0 است.

فصل ۵: شرط و تصمیم‌گیری

پاسخ آزمونک

1. **۱ — الف.** عبارت درست `سن بزرگتر یا مساوی ۱۸` است.
2. **۲ — ب.** ۷ با ۷ برابر است؛ پس «نابرابری» نادرست (False) است.
3. **۳ — ب.** تورفتگی بدنهٔ شرط را مشخص می‌کند.
4. **۴ — ب.** شرط اول نادرست، شرط دوم (۱۴ ≤ ۱۲) درست است.
5. **۵ — ج.** «و» یعنی هر دو.

تمرین ۱

```
بگذار سن برابر ۱۴
اگر سن کوچکتر از ۱۲
    چاپ -کودک-
وگرنه اگر سن کوچکتر از ۱۸
    چاپ -نوجوان-
وگرنه
    چاپ -بزرگسال-
```

خروجی

```
نوجوان
```

با سن‌های ۸ و ۱۴ و ۲۵ هم امتحان کنید.

تمرین ۲

```
بگذار متن۱ برابر بخوان
بگذار متن۲ برابر بخوان
بگذار الف برابر به_عدد با متن۱
بگذار ب برابر به_عدد با متن۲
اگر الف بزرگتر از ب
    چاپ الف
وگرنه
    چاپ ب
```

خروجی

ورودی: ۴ ، ۹

```
9.0
```

ورودی‌ها متن‌اند؛ پیش از مقایسه با `به_عدد` به عدد تبدیل شده‌اند. نتیجه اعشاری (9.0) چاپ می‌شود.

تمرین ۳

```
چاپ -دما چند است؟-
بگذار متن برابر بخوان
بگذار دما برابر به_عدد با متن
اگر دما بزرگتر از ۳۰
    چاپ -هوا گرم است-
وگرنه اگر دما بزرگتر یا مساوی ۱۵
    چاپ -هوا معتدل است-
وگرنه
    چاپ -هوا سرد است-
```

خروجی

ورودی: ۳۲

```
دما چند است؟
هوا گرم است
```

با ۲۰ و ۵ هم آزمایش کنید و پیش از اجرا خروجی را حدس بزنید.

تمرین ۴

```
بگذار متن برابر بخوان
بگذار عدد برابر به_عدد با متن
اگر عدد % ۲ == ۰
    چاپ -زوج-
وگرنه
    چاپ -فرد-
اگر عدد بزرگتر از ۱۰۰
    چاپ -بزرگ‌تر از ۱۰۰-
وگرنه
    چاپ -۱۰۰ یا کمتر-
```

خروجی

ورودی: ۱۲۷

```
فرد
بزرگ‌تر از ۱۰۰
```

دو شرط مستقل پشت سر هم نوشته شده‌اند؛ هر کدام جداگانه سنجیده می‌شود.

فصل ۶: حلقه و تکرار

پاسخ آزمونک

1. **۱ — ج.** بدنه ۳ بار اجرا می‌شود.
2. **۲ — ب.** تا وقتی شرط درست باشد.
3. **۳ — ب.** روی همهٔ اعضا می‌گردد.
4. **۴ — ب.** «بیرون».
5. **۵ — الف.** شرط همیشه درست می‌ماند.

تمرین ۱

```
تکرار ۵ بار
  چاپ -سلام-
```

خروجی

```
سلام
سلام
سلام
سلام
سلام
```

همان چیزی که در فصل ۱ با چهار دستور می‌نوشتیم، حالا دو خط شد.

تمرین ۲

```
بگذار ن برابر ۱
تاوقتی ن =< ۱۰
  چاپ ن
  بگذار ن برابر ن + ۱
```

خروجی

```
1
2
3
4
5
6
7
8
9
10
```

شرط را به `ن < ۱۱` هم می‌شود نوشت؛ نتیجه یکی است.

تمرین ۳

```
بگذار عدد برابر ۷
بگذار ی برابر ۱
تاوقتی ی => ۱۰
  چاپ عدد * ی
  بگذار ی برابر ی + ۱
```

خروجی

```
7
14
21
28
35
42
49
56
63
70
```

عدد را از کاربر هم می‌توانید بگیرید (به_عدد فراموش نشود).

تمرین ۴

```
بگذار مجموع برابر ۰
بگذار ن برابر ۰
تاوقتی ن < ۹
  بگذار ن برابر ن + ۱
  اگر ن % ۲ == ۰
    ادامه
  بگذار مجموع برابر مجموع + ن
چاپ مجموع
```

خروجی

```
25
```

۱ + ۳ + ۵ + ۷ + ۹ = ۲۵.

فصل ۷: تابع و استفادهٔ مجدد از کد

پاسخ آزمونک

1. **۱ — ج.** «بازگردان» نتیجه را برمی‌گرداند.
2. **۲ — ب.** ۵ × ۵ = ۲۵.
3. **۳ — ب.** الف و ب: دو ورودی.
4. **۴ — ب.** خوانایی و پرهیز از تکرار.
5. **۵ — ب.** متغیر محلی است.

تمرین ۱

```
تابع دو_برابر با عدد
    بازگردان عدد * ۲

چاپ دو_برابر با ۴
```

خروجی

```
8
```

نتیجه ۸ است.

تمرین ۲

```
تابع مکعب با عدد
    بازگردان عدد * عدد * عدد

چاپ مکعب با ۳
```

خروجی

```
27
```

۳ × ۳ × ۳ = ۲۷.

تمرین ۳

```
تابع پیام با نام
    بازگردان -سلام - + نام

چاپ پیام با -علی-
چاپ پیام با -مینا-
چاپ پیام با -پویا-
```

خروجی

```
سلام علی
سلام مینا
سلام پویا
```

ورودی و خروجی را با زبان ساده توضیح دهید: ورودی نام، خروجی جملهٔ سلام.

تمرین ۴

```
تابع اندازه با عدد
    اگر عدد < ۱۰
        بازگردان -بزرگ-
    بازگردان -کوچک-

برای عدد در [۵، ۱۵، ۱۰]
    چاپ اندازه با عدد
```

خروجی

```
کوچک
بزرگ
کوچک
```

ترکیب تابع، شرط و حلقه؛ ۱۰ «کوچک» است چون بزرگ‌تر از ۱۰ نیست.

فصل ۸: فهرست و فرهنگ

پاسخ آزمونک

1. **۱ — ب.** جایگاه‌ها از صفر شروع می‌شوند.
2. **۲ — الف.** تعداد اعضا ۳ است.
3. **۳ — الف.** `افزودن با فهرست و مقدار`.
4. **۴ — ب.** با کلید.
5. **۵ — ب.** هر مقدار برچسب دارد؛ فرهنگ.

تمرین ۱

```
بگذار رنگ‌ها برابر [-قرمز-، -سبز-، -آبی-]
چاپ رنگ‌ها[۰]
چاپ رنگ‌ها[۲]
```

خروجی

```
قرمز
آبی
```

عضو آخر شمارهٔ ۲ دارد.

تمرین ۲

```
بگذار عددها برابر [۲، ۴، ۶، ۸]
بگذار مجموع برابر ۰
برای عدد در عددها
    بگذار مجموع برابر مجموع + عدد
چاپ مجموع
```

خروجی

```
20
```

خروجی ۲۰.

تمرین ۳

```
بگذار دوستان برابر [-علی-، -مینا-]
افزودن با دوستان و -پویا-
چاپ دوستان
چاپ طول با دوستان
```

خروجی

```
['علی', 'مینا', 'پویا']
3
```

خروجی: فهرست سه‌عضوی و بعد عدد 3.

تمرین ۴

```
بگذار فرهنگ_لغت برابر فرهنگ
    کتاب برابر -book-
    قلم برابر -pen-
    میز برابر -table-
چاپ -کدام واژه؟-
بگذار واژه برابر بخوان
چاپ فرهنگ_لغت[واژه]
```

خروجی

ورودی: **قلم**

کدام واژه؟

pen

اگر واژه‌ای را وارد کنید که در فرهنگ نیست، پیام خطا می‌بینید.

فصل ۹: گرافیک و نقاشی با برنامه

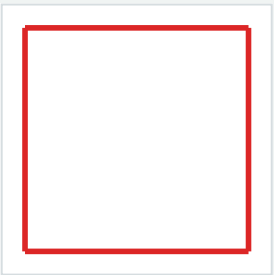
پاسخ آزمونک

1. **۱ — ب.** از وسط صفحه و رو به راست شروع می‌کند.
2. **۲ — ج.** زاویه‌های مربع قائمه (۹۰ درجه) است.
3. **۳ — الف.** `-رنگ قلم برابر -آبی`
4. **۴ — ج.** هر بار ۱۴۴ درجه.
5. **۵ — ب.** قالب SVG.

تمرین ۱

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -قرمز-
ضخامت قلم برابر ۳
مربع با ۱۲۰
```

تصویر ساخته‌شده

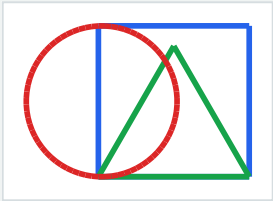


فایل را با `tirotir graphics` یا نمایش زندهٔ VS Code ببینید.

تمرین ۲

پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -آبی-
ضخامت قلم برابر ۳
مربع با ۸۰
رنگ قلم برابر -سبز-
مثلث با ۸۰
رنگ قلم برابر -قرمز-
دایره با ۴۰

تصویر ساخته‌شده

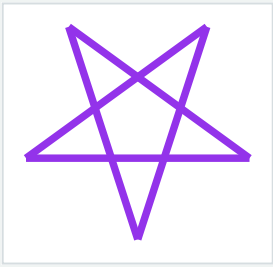


شکل‌ها از یک نقطه شروع می‌شوند و روی هم می‌افتند؛ با «قلم بالا» و «حرکت» جابه‌جایشان کنید.

تمرین ۳

پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -بنفش-
ضخامت قلم برابر ۴
تکرار ۵ بار
حرکت با ۱۲۰
چرخش با ۱۴۴

تصویر ساخته‌شده

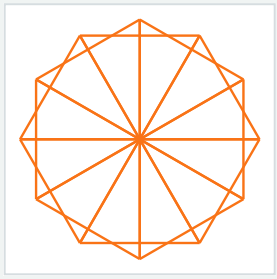


زاویهٔ ۱۴۴ درجه ستاره می‌سازد.

تمرین ۴

```
پنجره بازکن با ۸۰۰ و ۶۰۰
رنگ قلم برابر -نارنجی-
ضخامت قلم برابر ۲
تکرار ۱۲ بار
    مثلث با ۱۰۰
    چرخش با ۳۰
```

تصویر ساخته‌شده



۱۲ بار × ۳۰ درجه = یک دور کامل.

فصل ۱۰: تولید صدا و موسیقی ساده

پاسخ آزمونک

1. **۱ — ب.** تعداد لرزش در ثانیه.
2. **۲ — د.** نت‌ها: دو، ر، می، فا، سل، لا، سی.
3. **۳ — ب.** مدت نواختن به ثانیه.
4. **۴ — ب.** فایل `tirotir-note.wav`.
5. **۵ — ب.** صدای ملایم.

تمرین ۱

```
نت بنواز با -دو- و ۰٫۲
نت بنواز با -ر- و ۰٫۲
نت بنواز با -می- و ۰٫۲
نت بنواز با -فا- و ۰٫۲
نت بنواز با -سل- و ۰٫۲
نت بنواز با -لا- و ۰٫۲
نت بنواز با -سی- و ۰٫۲
```

نتیجه

فایل صوتی tirotir-note.wav ساخته شد.

بدون خروجی متنی؛ فایل `tirotir-note.wav` ساخته می‌شود.

تمرین ۲

```
بگذار نت‌ها برابر [-دو-، -می-، -سل-، -می-، -دو-، -می-، -سل-، -سل-]
برای نام_نت در نت‌ها
    نت بنواز با نام_نت و ۰٫۲
```

نتیجه

فایل صوتی tirotir-note.wav ساخته شد.

هر ۸ نت را بشمارید.

تمرین ۳

```
تابع جفت با الف و ب
  نت بنواز با الف و ۰٫۱۵
  نت بنواز با ب و ۰٫۱۵

تکرار ۴ بار
  جفت با -دو- و -سل-
```

نتیجه

فایل صوتی tirotir-note.wav ساخته شد.

تابع دو ورودی دارد؛ حلقه آن را تکرار می‌کند.

تمرین ۴

```
تکرار ۳ بار
  نت بنواز با -دو- و ۰٫۱
  نت بنواز با -دو- و ۰٫۱
  نت بنواز با -سل- و ۰٫۲
```

نتیجه

فایل صوتی tirotir-note.wav ساخته شد.

ریتم تکراری با یک حلقه.

فصل ۱۱: ساخت برنامهٔ دارای پنجره (رابط گرافیکی)

پاسخ آزمونک

1. **۱ — د.** حلقه ساختار برنامه است، نه جزء پنجره.
2. **۲ — ب.** «نمایش پنجره» پنجره را روی صفحه می‌آورد.
3. **۳ — الف.** `tirotir gui app.t` .
4. **۴ — الف.** Tkinter.
5. **۵ — ج.** رویدادها هنوز تکمیل نشده.

تمرین ۱

```
پنجره بساز با -برنامهٔ من- و ۴۰۰ و ۳۰۰
برچسب بساز با -سلام-
دکمه بساز با -بزن-
نمایش پنجره
```

نتیجهٔ بررسی

برنامه صحیح است.

اگر Tkinter و نمایشگر دارید با «اجرای GUI» ببینید.

تمرین ۲

```
پنجره بساز با -کارت معرفی- و ۴۰۰ و ۳۰۰
برچسب بساز با -نام-
ورودی بساز
برچسب بساز با -شهر-
ورودی بساز
دکمه بساز با -ثبت-
نمایش پنجره
```

نتیجهٔ بررسی

برنامه صحیح است.

ترتیب اجزا همان ترتیب نمایش است.

تمرین ۳

```
پنجره بساز با -کارت تبریک- و ۴۰۰ و ۲۴۰
برچسب بساز با -عید شما مبارک-
برچسب بساز با -سال نو پر از شادی-
دکمه بساز با -ممنون-
نمایش پنجره
```

نتیجهٔ بررسی

برنامه صحیح است.

متن‌ها را کوتاه و خوانا بنویسید.

تمرین ۴

```
پنجره بساز با -فرم- و ۵۰۰ و ۴۰۰
برچسب بساز با -نام-
ورودی بساز
چک‌باکس بساز با -عضویت-
منوی انتخاب بساز با -الف- و -ب- و -ج-
دکمه بساز با -ثبت-
نمایش پنجره
```

نتیجهٔ بررسی

برنامه صحیح است.

تفاوت طرح اولیه و نتیجه را توضیح دهید.

فصل ۱۲: بررسی برنامه و رفع خطا

پاسخ آزمونک

1. **۱ — ج.** خطای منطقی.
2. **۲ — الف.** نامی تعریف نشده است.
3. **۳ — ب.** check خطاهای زمان اجرا را نمی‌بیند.
4. **۴ — ب.** اول باید خطا را بازتولید کنیم.
5. **۵ — ج.** اطلاعات محرمانه را هرگز نفرستید.

تمرین ۱

```
بگذار نام برابر -سارا-
بگذار نمره برابر ۱۷
بگذار جداکننده برابر -: -
اگر نمره بزرگتر یا مساوی ۱۰
    چاپ -قبول-
وگرنه
    چاپ -مردود-
چاپ نام + جداکننده + به_متن با نمره
```

خروجی

```
قبول
سارا: 17
```

سه خطا: (۱) زیر «اگر» تورفتگی نیست؛ (۲) بعد از `+` متن ثابت نوشته شده است؛ (۳) عدد بدون «به_متن» به متن چسبانده شده است.

تمرین ۲

```
بگذار میانگین برابر (۱۲ + ۱۸) / ۲
چاپ میانگین
```

خروجی

```
15.0
```

پرانتز لازم بود.

تمرین ۳

```
بگذار مجموع برابر ۰
برای عدد در [۳، ۶، ۹]
    بگذار مجموع برابر مجموع + عدد
    چاپ مجموع
```

خروجی

```
3
9
18
```

خروجی ۳، ۹ و ۱۸.

تمرین ۴

```
چاپ -برنامه درست شد-
```

خروجی

```
برنامه درست شد
```

این تمرین گروهی است و پاسخ یکتا ندارد.

فصل ۱۳: ساخت برنامهٔ اجرایی با Tirotir Builder

پاسخ آزمونک

1. **۱ — ب.** گیرنده نیازی به نصب تیروتیر ندارد.
2. **۲ — ب.** روی خود ویندوز.

3. **۳ — ب.** برنامه باید درست و آزمایش‌شده باشد.
4. **۴ — الف.** نصب‌کنندهٔ تاریخی نسخهٔ ۱.
5. **۵ — ب.** منبع را معرفی و مجوز را رعایت کنید.

تمرین ۱

```
چاپ -نام شما چیست؟-
بگذار نام برابر بخوان
بگذار سلام برابر -سلام -
چاپ سلام + نام
```

نتیجهٔ بررسی

برنامه صحیح است.

پیش از ساخت EXE، بررسی را حتماً انجام دهید.

تمرین ۲

```
چاپ -نام شما چیست؟-
بگذار نام برابر بخوان
بگذار سلام برابر -سلام -
چاپ سلام + نام
```

خروجی

ورودی: **علی**

نام شما چیست؟

سلام علی

ورودی دلخواه است.

تمرین ۳

```
چاپ -برنامهٔ من آماده است-
```

خروجی

برنامهٔ من آماده است

tirotir.exe: خود زبان؛ program.exe: خروجی برنامهٔ شما؛ Setup: نصب‌کنندهٔ تاریخی نسخهٔ ۱.

تمرین ۴

```
چاپ -عدد اول؟-
بگذار متن۱ برابر بخوان
چاپ -عدد دوم؟-
بگذار متن۲ برابر بخوان
بگذار الف برابر به_عدد با متن۱
بگذار ب برابر به_عدد با متن۲
چاپ الف + ب
```

خروجی

ورودی: **۳ ، ۴**

عدد اول؟

عدد دوم؟

```
7.0
```

نتیجه ۷٫۰ است.

فصل ۱۴: پروژهٔ چندفایلی، قابلیت‌ها و ارائهٔ پروژهٔ پایانی

پاسخ آزمونک

1. **۱ — ب.** فایل `tirotir.toml`.
2. **۲ — الف.** بدون نام فایل در ریشه: `tirotir run`.
3. **۳ — ب.** پیش‌فرض خاموش است.
4. **۴ — الف.** حداکثر پنج دقیقه.
5. **۵ — ب.** ذکر منبع، اخلاق حرفه‌ای است.

تمرین ۱

```
چاپ -پروژهٔ من: بازی حدس عدد-
```

خروجی

```
پروژهٔ من: بازی حدس عدد
```

عنوان، ورودی، خروجی و مفاهیم را بنویسید.

تمرین ۲

```
چاپ -پروژهٔ دوفایلی من-
```

خروجی

```
پروژهٔ دوفایلی من
```

ساختار پوشه و tirotir.toml را از متن فصل بردارید.

تمرین ۳

```
بگذار فرصت برابر ۵
بگذار راز برابر ۴
تاوقتی فرصت < ۰
    بگذار متن برابر بخوان
    بگذار حدس برابر به_عدد با متن
    اگر حدس == راز
        چاپ -درست-
        بیرون
    بگذار فرصت برابر فرصت - ۱
چاپ فرصت
```

خروجی

```
ورودی: ۱، ۴
درست
4
```

با «بیرون» وقتی درست حدس زد از حلقه خارج می‌شویم؛ خروجی آخر، فرصت‌های باقی‌مانده است.

تمرین ۴

چاپ -پروژهٔ پایانی من-

خروجی

پروژهٔ پایانی من

پاسخ یکتا ندارد؛ شما طراح هستید!